r2-sim simulation methodology

Radiance 2 technical specification, document 3

r2-spec

Version 0.2, draft of 2026-10-16

Contents

1 Introduction **5**

1.1 Purpose . . . 5
1.2 Approach . . . 5
1.3 Scope . . . 5
1.4 Decisions not implemented yet . . . 5
1.5 Code map . . . 6
1.6 Conventions . . . 6

2 Three planes **7**

2.1 Global clock . . . 7
2.2 Interfaces . . . 7
2.3 Threads . . . 9
2.4 Modes . . . 9

3 Functional plane **10**

3.1 Backends . . . 10
3.2 Functional ops . . . 10
3.3 Worker . . . 11
3.4 Sync points . . . 11

4 Control plane **12**

4.1 Simulation loop . . . 12
4.2 Programs . . . 13
4.3 Costs of the Neutrino . . . 14
4.4 Model of the Neutrino tables . . . 14
4.5 Task issue . . . 16

4.6 Completion . . . 17
4.7 Messages . . . 17
4.8 Hierarchy . . . 19
4.9 Stalls, deadlocks and errors . . . 19

5 Resource plane **20**
5.1 Resources . . . 20
5.2 Slots and clock ratios . . . 21
5.3 Calendars . . . 22
5.4 Chunks . . . 23
5.4.1 Issue of transfer chunks . . . 23
5.5 Routing . . . 24
5.6 Charging a transfer chunk . . . 25
5.7 Charging a compute chunk . . . 26
5.8 Occupancy resources . . . 27
5.9 Latency . . . 27
5.10 Memory access of a program . . . 27
5.11 Culprits and delays . . . 28

6 PE models **28**
6.1 DMA engine . . . 29
6.2 Atlas . . . 29
6.3 Muon . . . 30

7 Utilization **30**
7.1 Definition . . . 31
7.2 Sources . . . 31
7.3 Calibration . . . 31
7.4 Mixed mode . . . 32

8 Cache model **32**
8.1 Configuration . . . 32
8.2 State . . . 32
8.3 Bookkeeping . . . 33
8.4 Flush . . . 34
8.5 Limits of the model . . . 34

9 SIMT modeling **34**

9.1 What a lane operation counts . . . 34
9.2 Utilization per kernel variant . . . 35
9.3 Calibration and validation . . . 35
9.4 Kernel families . . . 36

10 Configuration **36**

10.1 Files . . . 36
10.2 Names . . . 37
10.3 Addressing . . . 37
10.4 Keys . . . 37
10.5 Overrides . . . 40
10.6 Checks . . . 41
10.7 Shipped configurations . . . 41

11 Outputs **42**

11.1 Summary . . . 42
11.2 Statistics file . . . 42
11.3 Phase statistics . . . 43
11.4 Culprits and delays . . . 43
11.5 Tracing . . . 43
11.6 Assembly listings . . . 44
11.7 Diagnostics . . . 45

12 Extending the simulator **45**

12.1 A new opcode . . . 45
12.2 A new kind of PE . . . 46
12.3 The programs and their builders . . . 46
12.4 A new program . . . 47
12.5 A new backend or tracer . . . 47
12.6 Checks of a change . . . 48

13 Examples **48**

13.1 One DMA through a route . . . 48
13.2 One GEMM CTA as tasks . . . 50
13.3 The RTX PRO 6000 as a modeled system . . . 51
13.3.1 Hierarchy and parameters . . . 51

13.3.2 GPU semantics in the model 52
13.4 Runs of the listings of documents 1 and 2 54

1 Introduction

r2-sim is the simulator of Radiance 2. It runs Neutrino programs on a model of a device. It gives the time of every task, the use of every resource and, when a program needs them, data values. This document specifies how r2-sim models a device in time: the methods, the mechanisms and examples.

Document 1 specifies the Neutrino: the instructions (Section 5), the tables (Section 6), the task lifecycle (Section 8) and the protocols between Neutrinos (Section 9). Document 2 specifies the hardware of Radiance 2: the hierarchy (Section 2), the memory system (Section 3), the placement of the Neutrinos (Section 4), the processing elements (Section 5) and the programming model (Section 6). Each block there has a cost model that is independent of a simulator (Section 2.3). This document does not repeat them. It tells how the simulator puts the cost models in time, what it approximates or leaves out, and what a user must give it.

1.1 Purpose

r2-sim must do these tasks:

1. Estimate the time of a workload, for example a full forward pass of a model, without the computation of data values. This is possible when the control flow of the programs does not depend on data.
2. Run the same programs with data values, to check their function.
3. Mix fidelity per subtree of the hardware: no data, recorded data or computed data.
4. Run a sweep of hundreds of configurations in minutes.
5. Attribute each delay of a chunk to one resource.

1.2 Approach

- The device is a tree of nodes that a configuration file describes (Section 10). The code fixes no shape.
- The simulator is event driven. One global clock counts SM cycles.
- The unit of work is the task. The resource plane charges a task in chunks, for example one output tile of a matmul or one burst of a copy. The number of events is proportional to the number of chunks. It does not depend on the number of cycles.
- The model has three planes (Section 2). The control plane orders the work. The resource plane gives the time of each chunk. The functional plane computes the data values.
- A chunk occupies a part of a resource, in slots. Two flows can share a resource in the same cycle. The resource plane keeps whole slots and whole cycles, so that a chunk never moves a fraction of a beat.

1.3 Scope

Table 1 gives what the model includes and what it leaves out.

1.4 Decisions not implemented yet

Documents 1 and 2 have decisions that r2-sim does not implement yet. Table 2 gives each one and what r2-sim does today.

Table 1: The scope of the model.

Modeled	Not modeled
The Neutrino tables, costs and protocols.	Arbitration inside a chunk: a chunk holds a constant usage.
PEs as demand profiles: DMA engines, Atlas tiles, Muon tasks.	The Muon instruction set, warps, occupancy and register pressure.
The fabric: buses, links, read and write channels, beats, clock ratios, latencies, hash links.	Backpressure from a slower hop downstream (no head-of-line blocking on data).
Memory blocks with ports and latencies; banks.	DRAM banks, rows and refresh; coherence; the lane banks of the SMEM and their swizzle (Section 3.1).
A cache of a DRAM in front of its channels (Section 8).	Cache sectors, associativity, fill writes.
A memory of the type `cache`: a port and a latency on the path (Section 10.4).	Its hits, misses and capacity.
The data of each transfer on the read and write channels.	The read requests and the write acknowledgements: a transfer pays the latency of its route once (Section 5.9).
The scalar work of a program as the costs of its instructions (Section 4.3).	The RV64 code of the scalar core: a program is Rust code (Section 4.2).
The tensor memory of an Atlas as an accumulator buffer of output tiles (Section 6.2).	The operands in the tensor memory, and its ports.
Untracked tasks, the fence (`wait 0`), the fence at the return of a program, and the check of the null address (Section 3.1 and Rule N2.1).	A finer fence, for example per CTA position: the `kernels` program keeps store records (Section 13.3.2).
	Power and area.

Table 2: Decisions of documents 1 and 2 that r2-sim does not implement yet.

Decision	r2-sim today
The Atlas reads its operands from the tmem: in the 2p family 256 KiB in four banks with a read port and a write port of 128 B/cycle each, filled by the DMA engine; the partial output tiles of a chain live there (Section 5.2).	The Atlas reads its operands from the SMEM; the accumulator buffer holds the C tiles.
Four Muons per SM in the 2p family, each with 16 BF16 or 8 FP32 lane operations per cycle (Section 5.3).	One Muon per SM with 256 lane operations per cycle and one count of lane operations per task.

1.5 Code map

The simulator is the Rust workspace `r2-sim`. Table 3 gives the crates and what each one holds.

1.6 Conventions

- A cycle is an SM cycle. A rate is per cycle of the clock of its block (Section 5.2). Latencies and costs are in SM cycles.
- An element has a dotted name from the root, for example `root.gpc0.sm1.smem`. The shipped configurations call a cluster `gpc` and an SM `sm`. The text uses the terms of the glossary (cluster, SM) and the configured names in code font.
- Names of code (functions, events, fields) are in code font. They help a reader who extends

Table 3: The crates of r2-sim.

Crate	Content
`nu-model`	Addresses, address sets, hash rewrites, the dependency tables of one Neutrino, the requester side of the dependency protocol. It has no notion of time.
`sim-core`	The event loop, the control plane, the resource plane, the routing tables, the demand profiles, the configuration loader, the cache model, the interface and worker of the functional plane, the tracer interface, the assembly listings.
`func`	The Trace and Numeric backends of the functional plane.
`nu-prog`	The programs: `adamoe`, `prefill`, `forward`, `micro` and `kernels`.
`r2sim`	The command line, the statistics file and the trace writers.
`python/r2sim`	Figures and a checker of raw dumps.

the simulator. They are not part of the method.
- The terms are those of the glossary (Section 4). Bold type marks a term where this document defines it.

2 Three planes

r2-sim splits the model of a device into three planes. Each plane has one job and its own state (Figure 1):

- The control plane orders all work in time. It runs the programs, the Neutrino model, the task lifecycle, the PE actors and the messages (Section 4).
- The resource plane answers one question: "this demand, at or after cycle t, ends when?" It keeps the calendars of the resources and records what it grants (Section 5).
- The functional plane computes data values (Section 3).

The planes meet at a few narrow interfaces. A user can change one plane, for example the backend of the functional plane, and keep the other two.

2.1 Global clock

The simulator has one clock. Its unit is the SM cycle, as an unsigned 64-bit integer. Every plane stamps its work with this clock.

A block that runs on another clock declares a clock ratio (`clock`): the frequency of the block over the SM frequency, as an integer or a fraction `p/q`. Its rates are per cycle of its own clock. The resource plane converts them to slots per SM cycle (Section 5.2). The model holds no absolute frequency. The key `global.clock_ghz` converts cycles to seconds in reports only.

Rule S2.1 (one clock). Every latency, cost and event time must be in SM cycles. Only a rate can refer to the clock of its block.

2.2 Interfaces

Table 4 lists every interface between the planes. The control plane calls the other two planes. The resource plane and the functional plane never call the control plane and never create events.

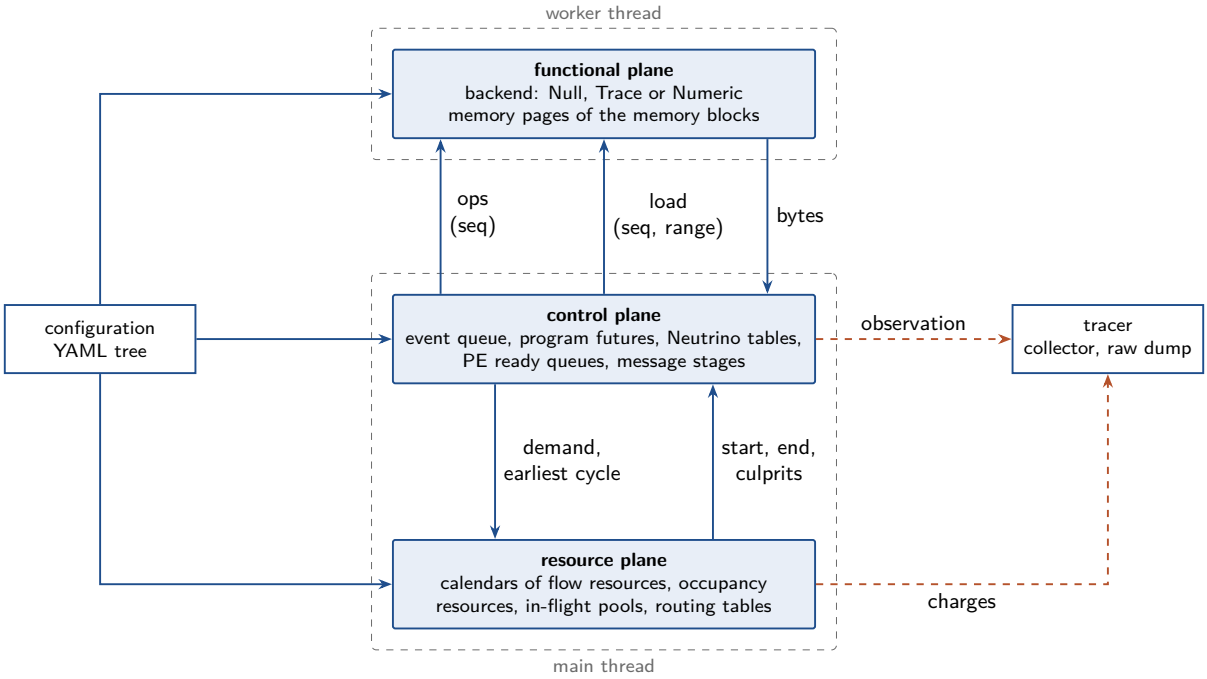


Figure 1: The three planes, their state and their interfaces. Solid arrows carry requests and answers. Dashed arrows are observation points: they carry no answer back.

Table 4: The interfaces between the planes.

From	To	Content
control	resource	A compute chunk: its demands and its earliest cycle (`reserve`). A transfer chunk: its hops, the beat and rate of its master, its earliest cycle and its pool data (`reserve_path`). One hop of a message (`reserve_message`). The holds of a task (acquire, release).
resource	control	The grant: start cycle, end cycle, culprit and queue culprit. A hold that does not fit puts the PE into the FIFO of the occupancy resource.
control	functional	One functional op per issued task and per store, with a sequence number. The call does not wait for the worker.
control	functional	A load: the sequence number of the last writer, the address and the length. The call waits for the bytes.
functional	control	The bytes of a load. An error of the backend.
control, resource	tracer	Observation points (Section 11.5). A tracer returns nothing.

Rule S2.2 (no data in timing). The control plane and the resource plane must not read data values, except at a sync point (Section 3.4). A profile must be a function of the descriptor and the configuration only.

Note. A shape that depends on data, for example the number of tokens that an expert receives, enters a descriptor through the program. The program reads the value with a load and writes it into the next descriptor. The load is the one point where the data affects the time.

Rule S2.3 (determinism). The result of a run must depend only on the configuration, the program, its parameters and the recorded data. It must not depend on the speed of the worker thread, on the host, or on a tracer.

The event queue orders events by cycle, then by insertion order. The functional plane cannot change the order of events (Section 3). A tracer cannot change it either (Rule S11.1). These three facts give Rule S2.3.

2.3 Threads

The control plane and the resource plane run on the main thread. The functional plane runs on one worker thread when its backend keeps data. The two threads communicate through a bounded queue of functional ops and one slot for a load (Section 3.3). A backend without data (Null) runs on the main thread and needs no worker.

2.4 Modes

A mode is a choice of settings. It is not a separate code path: the control plane always orders the work. Table 5 gives the modes.

Table 5: Modes of a run.

Mode	Resource plane	Backend	Use and selection
timing only	full	Null or Trace	Latency estimates without data. The `r2sim` command line uses Trace, with the records of `-trace` or `-synthetic`, and Null for the `kernels` program.
functional check	zero time	Numeric	Function of a program. The library call `Sim::set_zero_time` selects zero-time mode.
full	full	Numeric	Data and time together, for validation.
mixed	full	per subtree	The key `fidelity` (null or numeric) of a node sets whether the memory blocks of its subtree keep data.

In **zero-time mode** every charge, latency and Neutrino cost takes zero cycles. The control plane still issues each task only after its dependencies, so the functional plane gets a valid sequence of ops (Section 3.2). For a race-free program, a functional check in zero time and a timed run with the Numeric backend give the same data.

Note. A backend that drives an RTL simulation of a subtree is part of the design (Section 7.4).

The code does not have it. The command line has no option for zero-time mode or for the Numeric backend; tests and library users select them.

3 Functional plane

The functional plane computes data values. It does not read the clock and it does not change the order of events. It affects the time of a run at one place only: the value that a load returns can change what the program does next (Section 3.4). The main thread waits for that value in wall time; the simulated clock does not move during the wait.

3.1 Backends

A **backend** implements the functional plane. A run has one backend. Table 6 gives the three backends and what each one does with a functional op and with a load.

Table 6: The backends of the functional plane.

Backend	On a functional op	On a load
Null	Discards the op.	Error: the run has no data, so a program must not load.
Trace	No computation. A DMA copies bytes. A store writes its bytes. A task whose label has a record writes the recorded bytes at the base of its destination range. Another compute task writes nothing.	Returns the bytes in memory.
Numeric	A DMA copies bytes. A store writes its bytes. An Atlas matmul computes C in 32-bit floating point from bf16 or f32 operands, with accumulation and split operands. A Muon task writes nothing.	Returns the bytes in memory.

The Trace and Numeric backends keep the same memory:

- One sparse store per memory block, in pages of 64 KiB. A page comes into existence at its first write. A byte that no op wrote reads as zero.
- A write into the sync-only space has no effect.
- A memory block in a subtree of fidelity null (Section 10.4) drops writes and rejects loads. A DMA from such a block copies nothing. This lets a model keep data only in the subtrees where programs read data.
- A task must write only inside its destination range. A DMA whose destination address is outside the destination range of its task stops the run.

A trace file holds records of a label and bytes. The command line reads it with `-trace FILE`, or makes records for the router of the action expert with `-synthetic SEED`. A label is a number that the program writes into a descriptor. It has no hardware meaning.

3.2 Functional ops

The control plane emits a **functional op** at two points:

- at the issue of a task on a PE: the op carries a copy of the descriptor;
- at the end of the memory access of a store of a program: the op carries the bytes.

A task without a PE (a `join` or a `spawn`) emits no op. Every op has a sequence number (1, 2, 3, ... in the order of emission), the cycle of emission, the Neutrino and the task.

Rule S3.1 (issue order). The control plane must emit the op of a task at its issue. The worker must apply the ops in the order of their sequence numbers.

A task issues only after the producers of its dependencies complete, and each producer emitted its op at its own issue. So the sequence of ops is a valid sequential execution of the dependency graph. The tables track read-after-write only (Section 6). Write-after-read and write-after-write order are the program's job: a new writer of a range depends on the readers of the old value, so its op follows their ops.

3.3 Worker

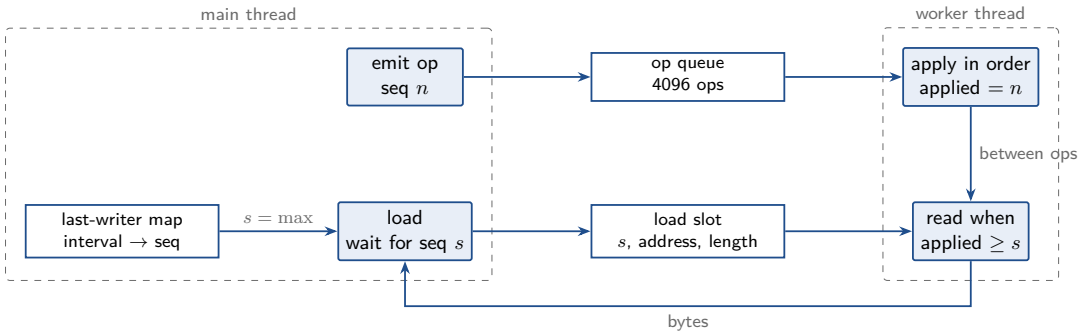


Figure 2: The main thread and the worker. Ops go through a bounded queue in order. A load goes through its own slot and the worker serves it as soon as it has applied the last writer of the range.

The worker is one thread (Figure 2):

1. The main thread puts ops into a queue of 4096 ops. When the queue is full, the main thread waits. This backpressure costs wall time only.
2. The worker takes the ops in order and applies them to its backend. It records the sequence number of the last op that it applied.
3. A load does not go through the queue. The main thread puts the load into a slot: the sequence number of the last writer, the address and the length. Then it waits.
4. Between two ops, the worker checks the slot. When it has applied the sequence number of the load, it reads the bytes and replies, also if more ops are in the queue.
5. The first error of the backend stops the worker. The main thread checks a flag every 1024 events and at the end of the run. The error report gives the sequence number, the cycle, the Neutrino and the task of the op, so it is the same in every run.

3.4 Sync points

A program that needs a value calls `load`. A load is a **sync point**. The control plane does these steps:

1. It charges the cost of the load on the scalar resource of the Neutrino (Section 4.3).

2. It charges the memory access: one transfer per memory block that the range touches, from the block to the Neutrino (Section 5.10).
3. At the end of the access, it does the race check (below).
4. It finds the sequence number of the last writer: the largest one that the last-writer map holds for any byte of the range.
5. It puts the load into the slot of the worker and waits for the bytes.
6. It gives the bytes to the program. The program continues at the same cycle.

The last-writer map holds, per address interval, the sequence number of the last op that wrote the interval. A task enters the map at its completion, with the sequence number of its op. A store enters it at the end of its access. A `join` enters it with the number 0.

Rule S3.2 (no race at a load). A program must not load a range while a task that writes the range is issued and not complete. The control plane checks this in the tables of the owner of the range. By default a race stops the run; a setting changes it to a warning.

With Rule S3.2 the value of a load does not depend on the speed of the worker. Every completed writer of the range has a sequence number at most equal to the number that the load waits for. A writer that is issued and not complete is a race. A writer that is not issued has no op yet. The main thread waits during the load, so no new op can come.

Example. The `adamoe` program models a mixture-of-experts layer. The router is a matmul at the home SM with a label. The Trace backend writes the recorded router output (one expert number per token) into its destination. The driver program then does this (shortened from `nu-prog`):

```
self.flush(ctx).await;                    // invoke the tasks of the window
if let Some(k) = self.g.key(&rout) {
    ctx.wait_key(k).await;                // the router output is done
}
ctx.load(Addr(rout.base), p.tokens).await  // the sync point: the bytes
```

With the bytes, the driver counts the tokens of each expert and invokes the tasks of the experts that received tokens. A different record gives a different set of tasks and a different time.

4 Control plane

The control plane orders all work in time. It runs the programs, the model of each Neutrino, the lifecycle of each task, the PE actors and the messages between Neutrinos. It asks the resource plane for the time of each piece of work and turns the answers into events.

4.1 Simulation loop

An **event** is a cycle, a sequence number and an action of the control plane. The event queue is a priority queue. It orders events by cycle, then by sequence number. The sequence number is the insertion order. Two events at the same cycle run in the order in which the control plane made them. The loop (Figure 3) does these steps:

1. Take the event with the smallest cycle and sequence number.
2. Set the clock to the cycle of the event.
3. Every 16 events, remove the parts of the calendars and pools before the current cycle (Section 5.3).

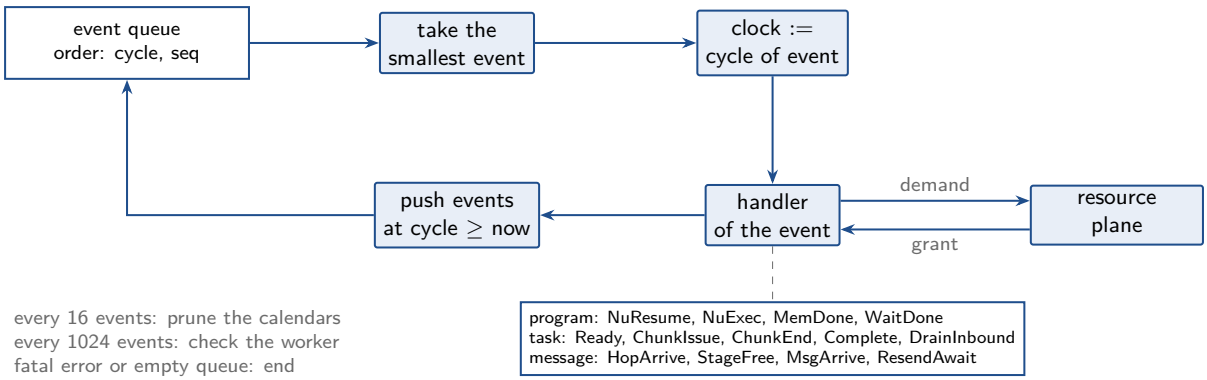


Figure 3: The simulation loop. Handlers ask the resource plane for grants and push new events at the current cycle or later.

4. Give the event to its handler. A handler changes the state, asks the resource plane for grants and pushes new events.
5. Every 1024 events, check the error flag of the worker.
6. Stop at the first fatal error, or when the queue is empty.

At the end, the control plane checks for a deadlock (Section 4.9) and collects the statistics (Section 11).

Rule S4.1 (no event in the past). A handler must push each new event at the current cycle or later.

Table 7 gives the events. A handler that needs a later action asks the resource plane for its time and pushes the event for that time.

Table 7: The events of the control plane.

Event	Action
`NuResume`	Poll the program of a Neutrino. Take its next request and charge its cost.
`NuExec`	The cost of the request is charged: execute the request.
`MemDone`	The memory access of a load or a store ends: do the sync point or emit the store.
`WaitDone`	A local wait ends, after the resolution delay.
`Ready`	A task is ready, after the resolution delay.
`ChunkIssue`	Issue the next chunk of a transfer.
`ChunkEnd`	A chunk ends (for a transfer without a cap: all its chunks).
`Complete`	The completion of a task reaches the tables, after the completion cost.
`DrainInbound`	Move forwarded invokes from the inbound invoke table into the scoreboard.
`HopArrive`	A message reaches the stage at the end of its hop.
`StageFree`	A message ends its transmission out of a stage: the stage has room again.
`MsgArrive`	A message reaches its receiver.
`ResendAwait`	Send again an Await that the owner rejected.

4.2 Programs

A program is an asynchronous Rust function over a context, `NuCtx`. Each hardware operation is a call on the context that the program awaits. The control plane drives the program as a coroutine. There is no asynchronous runtime. One call goes through these steps:

1. `NuResume`: the control plane polls the program. The call puts a request into the mailbox of the Neutrino and returns "pending". When the program returns, the Neutrino becomes idle (Section 4.8).
2. The control plane charges the cost of the request on the scalar resource of the Neutrino (Section 4.3) and pushes `NuExec` at the end of the charge.
3. `NuExec`: the control plane executes the request: it changes the tables, sends messages or starts a memory access.
4. The control plane puts the answer into the mailbox and pushes `NuResume` at the current cycle. A request can wait first: on a full table (a stall), on a key, or on a memory access.

Rule S4.2 (one call). A program must have at most one call in progress. It must await only calls of its own context.

Control that depends on data is ordinary Rust code over the bytes that a load returns. A program is also a value: a parent gives a program to a child with `spawn_child` (Section 4.8). The programs of `nu-prog` use this to build a hierarchy of drivers (Section 12.3). Document 1 gives the meaning of the instructions (Section 5). The scalar core of Radiance 2 runs RV64 code with the Neutrino instructions (Section 6.1). r2-sim does not run such code yet. A later front end will run it on the same interface. Table 8 gives the calls and their cost in the model.

4.3 Costs of the Neutrino

A Neutrino has two flow resources of kind `ScalarOps`: the scalar resource (`<node>.scalar`, rate `scalar_rate`, default 1) and the completion port (`<node>.complete`, rate `complete_rate`, default 1). Program calls use the scalar resource. Completions use the completion port, at $c_{complete}$ (default 4) cycles per task. So completions and program calls do not delay each other. The delay $c_{resolve}$ (default 0) uses no resource: it is the time from the last dependency of a task to its ready state, and from a done key to the wake of a local wait. The keys of the costs are in the `neutrino` section of a node.

Each call that is a request is one instruction of document 1 (Table 5), with three exceptions. `mark` and `cache_flush` exist only in the simulator. `plan(k)` is the instruction `cycles` of $k \times c_{plan}$ cycles, and it makes no request when that is 0. The assembly text prints `mark` and `cache_flush` as lines of their own (Section 11.6).

The assembly cost of a descriptor is

$$c_{desc} = c_{base} + c_{dep} \times deps + c_{word} \times \lceil wire/8 \rceil,$$

with the defaults $c_{base} = 8$, $c_{dep} = 3$ and $c_{word} = 1$. The wire size of a descriptor is 20 bytes (opcode, PE, destination range), plus 8 bytes per dependency, plus its arguments: 32 for a DMA, 52 for a matmul (and 8 per split operand), 24 for a Muon task (32 with a source address), 0 for a join. A spawn counts 24 bytes plus 8 per dependency. The label is not sent.

Example. A DMA descriptor without dependencies has $20 + 32 = 52$ wire bytes, so 7 words. With the defaults, $c_{desc} = 8 + 7 = 15$ cycles and the invoke takes $15 + 8 = 23$ cycles of the scalar resource. A program that calls `invoke` at cycle 0 has its task in the tables at cycle 23 (Section 13.1).

4.4 Model of the Neutrino tables

The crate `nu-model` holds the tables of one Neutrino (Section 6): their entries, capacities and lookups. It has no notion of time. Each change returns its effects: tasks that became ready, waits

Table 8: The calls of a program and their costs on the scalar resource (defaults in parentheses).

Call	Effect	Cost
`invoke(desc)`	Submits a task; returns a handle.	$c_{desc} + c_{invoke}$ (8)
`wait(addr, v)`, `wait_unversioned(addr)`, `wait_key(key)`	Blocks until a key is done.	c_{wait} (2)
`invoke_untracked(desc)`	Submits an untracked task (Section 3.1); returns nothing. `invoke` of an untracked descriptor is a program error.	$c_{desc} + c_{invoke}$ (8)
`fence()`	The instruction `wait` on the key 0: blocks until every untracked task of the program is done.	c_{wait} (2)
`version(addr, v)`	Sets the version counter of an own range.	$c_{version}$ (1)
`vreset_all()`	Clears the version table.	$c_{version}$ (1)
`set_epoch(e)`	The instruction `epoch`: sets the epoch and clears the version table (Section 7.3).	$c_{version}$ (1)
`vreset(addr)`	Frees one version entry; a VersionReset message for a range of another Neutrino.	c_{vreset} (1)
`load(addr, len)`	Reads bytes: a sync point.	c_{load} (1), then the access
`store(addr, bytes)`	Writes bytes.	c_{store} (1), then the access
`spawn_child(...)`, `spawn_named(...)`	Starts a program on a direct child, in epoch 0.	$c_{desc} + c_{spawn}$ (16)
`spawn_in_epoch(...)`	Starts a program on a direct child in an epoch (Section 7.3).	$c_{desc} + c_{spawn}$ (16)
`cycles(n)`	Scalar work of the program.	n
`plan(k)`	The planning of k tasks.	$k \times c_{plan}$ (0)
`mark(name)`	Records a mark (Section 11.3).	0
`cache_flush()`	Empties every cache (Section 8.4).	0
`now()`, `id()`, `children()`	Read the clock, the Neutrino of the program and its children.	none: not a request
`epoch()`	Reads the epoch of the Neutrino.	none: not a request

that ended, remote waiters to notify, entries that retired. The control plane turns the effects into events and messages.

An invoke enters the tables of the owner of its destination range. If another Neutrino owns the range, the invoke is forwarded (Section 4.7). At the owner, the control plane does these steps:

1. It checks that the PE of the descriptor belongs to the owner.
2. It splits the dependencies by owner. Each remote dependency needs an entry of the outbound dep table, counts as one unit of the depcount and sends an Await (Section 9).
3. It plans the invoke against the tables and changes nothing. The plan checks the range, a free scoreboard entry and the issue headroom. It finds the version of a versioned destination and looks up each local dependency. It also finds the pending entries that the task adopts and the done unversioned entries that it replaces.
4. It commits the plan. The task enters the scoreboard with its depcount. A task with depcount 0 is ready at once.

A lookup gives the tags of the producers in flight, “done”, or a new pending entry for a key whose producer is not invoked yet. Each table entry keeps a list of its waiters. A completion wakes the waiters of its entry. This is equal, as seen from outside, to a broadcast of the tag of the producer.

A failed plan leaves no trace. Table 9 gives what happens.

Table 9: A task that does not fit the tables of its owner.

Condition	Local invoke	Forwarded invoke
The scoreboard is full.	stall	stays in the inbound invoke table
No issue headroom.	stall	stays
No room in the outbound dep table for its remote dependencies.	stall	stays
The pending table is full.	fatal error	stays
The version table is full.	fatal error	stays
More remote dependencies than the outbound dep table holds; a version base with two lengths; a dependency on its own key.	fatal error	fatal error

A forwarded invoke that does not fit stays at the head of the inbound invoke table. The invokes behind it wait, so the owner accepts the invokes of each requester in program order. The control plane tries the head again after each event that frees an entry.

Note. A program infers the version of a forwarded invoke (Section 7) and can write the inferred version into the descriptor. A debug build compares it with the version that the owner assigns and stops the run on a difference. A release build does not check it.

4.5 Task issue

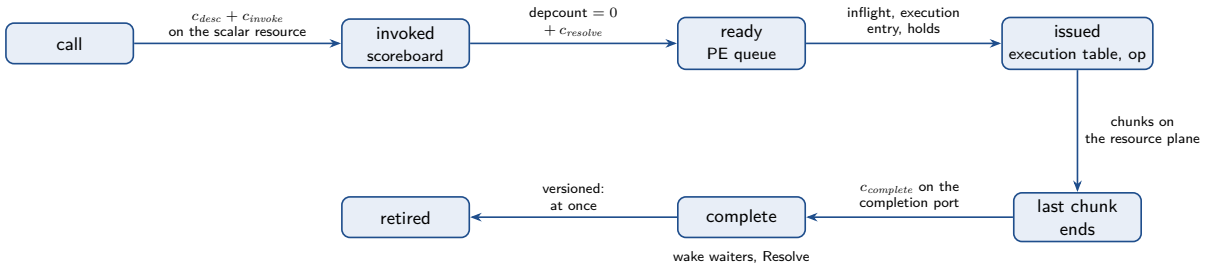


Figure 4: The life of one task in the control plane, with the costs and the conditions between the steps.

Figure 4 shows the life of one task. When the depcount of a task falls to 0, the task is ready after the delay $c_{resolve}$. A ready task with a PE enters the ready queue of its PE. The queue is in order: a task does not pass the task in front of it. The control plane then issues tasks from the head of the queue while these conditions hold:

1. The PE runs fewer tasks than its `inflight` limit.
2. The execution table of the Neutrino of the task has a free entry. Otherwise the PE waits for one.
3. The profile of the task exists. The control plane computes it at the first try (Section 6). The cache bookkeeping runs at this point (Section 8).
4. Each hold of the profile fits its occupancy resource. Otherwise the PE waits in the queue of that resource.

At issue, the task enters the execution table, takes its holds and emits its functional op. Its scoreboard entry is free, so a stalled request of the Neutrino tries again. Then the chunks of the

task start:

- A task without chunks completes at once.
- A transfer streams its chunks at the beat rate of its engine (Section 5.4.1).
- A compute task requests `outstanding` chunks ahead (default 1). Each chunk end requests the next chunk.

A task without a PE does not use a ready queue. A `join` enters the execution table and completes after $c_{complete}$. A `spawn` enters the execution table and sends Start to its child.

4.6 Completion

When the last chunk of a task ends, the PE runs one task less. The control plane charges $c_{complete}$ on the completion port and pushes `Complete` at the end of the charge. At `Complete` the tables mark the task done, and the control plane applies the effects:

- Local consumers count down. A consumer with depcount 0 is ready after $c_{resolve}$.
- A local wait of the program on the key ends after $c_{resolve}$.
- Each remote waiter gets a Resolve message.
- A versioned task retires at completion. An unversioned task retires when a new producer of its range replaces it, or when its program returns.
- The holds of the task go back to their resources. Waiting PEs try again.
- When an entry frees, stalled requests and PEs that wait for the execution table try again.
- The last-writer map records the destination range of the task.

4.7 Messages

The protocols between Neutrinos (Section 9) use messages on the control fabric. A message belongs to one of three classes. Each class has its own **channel**, so a message of one class never waits for a message of another class.

Class	Messages
invoke	Invoke, VersionReset; with ticket flow also InvokeAck, RetryAck, Reject, PCrdGrant
dependency	Await, AwaitAck, Resolve
hierarchy	Start, Return

Size. An Invoke carries the descriptor: its wire bytes (Section 4.3). Any other message has one beat. A message has $\lceil bytes/\texttt{msg_bytes} \rceil$ beats, at least 1 and at most 4. A larger message stops the run. The default of `msg_bytes` is 64.

Route. A message takes the control route from the attachment of the sender on its control bus to the attachment of the receiver. Its hops are the rated ports of the control crossbars and the rated directions of the control links (Section 5.5). The latency of a hop without a rate adds to the hop before it, or to the lead of the route. The key `global.msg_latency` adds cycles to the lead of every protocol message. Start and Return do not get it.

Channel. The channel of a class (Figure 5) has a source queue at the sender, one **stage** at the end of each rated hop, and the receiver. The source queue of the invoke class is the outbound invoke table of the sender (`outbound_invokes` messages). A full outbound invoke table stalls

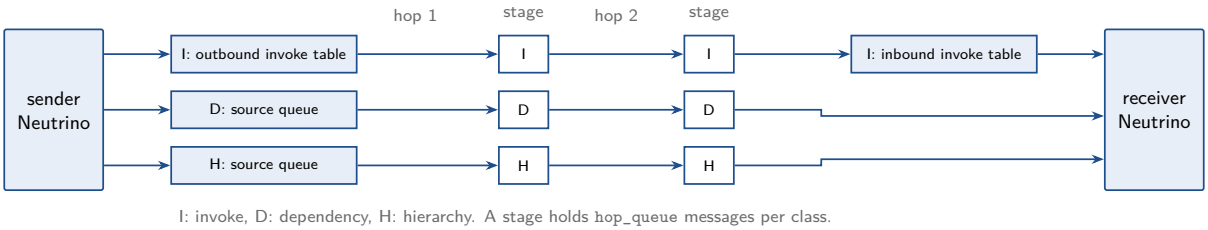


Figure 5: The channels of the three classes from a sender to a receiver: the source queues, one stage per class at the end of each hop, and the receiver.

the program. The source queues of the other classes have no limit. A hop stage holds `hop_queue` messages (default 1). The receiver of an Invoke is the inbound invoke table of the owner. A message moves by these rules (ready and valid):

1. The head of a stage goes onto the next hop only when the stage at the end of that hop has room. It takes that room at the start of its transmission.
2. The transmission is a charge on the hop. The message holds one slot, or the full capacity of a hop with less than one slot, for its beats. Its beat is the smaller of the beat of the sender and the beat of the hop. The hop stores the whole message and then forwards it, so the hops before do not narrow it.
3. At the end of the transmission, the stage that the message left has room again (`StageFree`). After the latency of the hop, the message enters the next stage (`HopArrive`).
4. A stage never changes the order of its messages. A head that cannot move blocks the messages behind it. Blocked stages continue in the order in which they blocked.
5. At the receiver, an Invoke needs a free entry of the inbound invoke table. It reserves the entry when it leaves the last stage.

A stage holds a message from the start of its transmission into the stage until the message leaves the stage. So one class carries at most `hop_queue` messages over a hop in the time of one transmission, the hop latency and the next transmission.

Example. The `micro` program in mode `invoke` on the 2p configuration: the device program invokes a `join` whose destination is in the sync-only space of `root.gpc0.sm0`. The descriptor has 20 wire bytes, so $c_{desc} = 11$ and the call takes 19 cycles of `root.scalar`. At cycle 19 the invoke is forwarded as an Invoke of one beat of 64 B. Every control hop moves 8 B per cycle, so the beat takes 8 cycles on each hop:

Hop	Slots	Latency	Window
`root.cbus.neutrino.in`	1	0	[19, 27)
`root.cbus.gpc0.clcbus_to_cbus.out`	1	0	[27, 35)
`root.gpc0.clcbus_to_cbus.ba`	1	20	[35, 43)
`root.gpc0.clcbus.clcbus_to_cbus.in`	1	0	[63, 71)
`root.gpc0.clcbus.sm0.smcbus_to_clcbus.out`	1	0	[71, 79)
`root.gpc0.sm0.smcbus_to_clcbus.ba`	1	10	[79, 87)
`root.gpc0.sm0.smcbus.smcbus_to_clcbus.in`	1	0	[97, 105)
`root.gpc0.sm0.smcbus.neutrino.out`	1	0	[105, 113)

The message arrives at cycle 113, after its last hop. The `join` enters the tables of the SM at 113, issues at 113 and completes at 117 after $c_{complete}$. The Invoke of a second call reaches the link at cycle 54 and starts on it at cycle 71: the first message holds the stage at the end of the link from

cycle 35 to cycle 71.

At the receiver. An Invoke enters the inbound invoke table and the owner moves it into its tables (Section 4.4). An Await gets a lookup at the owner and an AwaitAck: done, registered or reject. The requester sends a rejected Await again after `t_retry` cycles. A Resolve satisfies a remote dependency. A VersionReset frees one version entry. Start and Return are the hierarchy protocol (Section 4.8).

Ticket flow. The setting `global.flow: ticket` selects a second scheme for comparison (Section 9). The owner answers each accepted request with InvokeAck, and the requester keeps each request in its outbound invoke table until the answer. At a full inbound invoke table the owner answers RetryAck and sets a ticket bit for the requester. It then rejects the later requests of that requester with Reject, and later grants room with PCrdGrant. The requester sends its requests again in program order. Ready and valid is the default.

4.8 Hierarchy

A call of `spawn_child` makes a task with the opcode `spawn` in the tables of the parent. Its destination is a range in the sync-only space of the parent (Rule N2.5; a spawn with its done range elsewhere stops the run), and it can have dependencies. When the task issues, the parent sends Start with the program to the child. The child must be a direct child of the parent and must be idle. The start clears the version table of the child and sets its epoch register (Section 7.3). When the program of the child returns, the child waits for its untracked tasks (Rule N8.1), then retires its done unversioned entries and sends Return. At Return the parent charges $c_{complete}$ and the spawn task completes. A wait on the destination range of the spawn task therefore waits for the child program.

4.9 Stalls, deadlocks and errors

A request that does not fit a full table stalls: the control plane keeps the request and tries it again when an entry frees. A program stalls on these conditions:

- a full scoreboard, or no issue headroom;
- a full outbound dep table, for the remote dependencies of an invoke or a wait;
- a full outbound invoke table.

The statistics count the stall cycles per Neutrino.

The control plane detects a deadlock in two ways:

- When it sends an Await again: no task issued, no task completed and no message moved for `livelock_cycles`, and no PE runs. The default of `livelock_cycles` is 10 times the largest `t_retry` plus the longest route latency.
- At the end of the run: a message is still in a stage, an invoke is still in an inbound invoke table, or a program has not returned.

A deadlock is a fatal error. Its report gives the occupancy of every table and every stage that holds messages, each program that has not returned with what it waits for, and the open pending entries.

A fatal error stops the run. The statistics keep the first error, and they stay valid up to its cycle. These are the fatal errors:

- a software error of the tables, for example a full pending table at a local invoke;
- a range that no Neutrino owns;
- a race at a load (Rule S3.2);
- an error of the backend;
- an operation that a program must not do, for example an invoke on a PE of another Neutrino or a spawn on a busy child;
- a deadlock.

The errors of the tables have the names of document 1 (Table 14). Table 10 gives the errors and the warnings that only r2-sim has.

Table 10: The errors and warnings of r2-sim in addition to the errors of the Neutrino.

Name	Cause
`Misaligned`	A destination range whose base or end is not a multiple of the granule. A program of r2-sim gives its ranges in bytes; the ranges of the architecture count granules (Section 2.3).
`BaseConflict`	Two bounds or two masks for one versioned base address (Rule N7.3). r2-sim records the bound and the mask of each versioned base; the version table of the architecture holds neither.
`Unproduced`	A lookup finds a key of an earlier epoch done, but its base never reached the count of the key in that epoch on the owner (Rule N7.6). r2-sim records the highest count of each base in each epoch; the architecture keeps no such record.
`Invalid`	Also: a wrong version inference, in a debug build (Section 4.4).
`Func`	An error of the functional backend, for example a write outside the destination range (Section 3.1).
`Deadlock`	No progress (see above).
`MultipleCovers` (warning)	One lookup binds two or more unversioned producers that execute at the same time: writers that the program did not order (Section 7.5).
`UnversionedWait` (warning)	An unversioned key and a versioned write meet on one range: the key never matches the write. A frequent cause is the handle of a forwarded invoke (version 0) used as a key.
`Race` (warning)	A load of a range that an issued task writes, when the race check is set to warn (Section 11.7).

5 Resource plane

The resource plane answers one question of the control plane: “this work, at or after cycle t, ends when?” It keeps a calendar for each flow resource, gives each chunk a time, and records what it gives. It never makes events. This section gives the resources, the slots, the chunks, the routes, the rules that charge a chunk, and the latencies.

5.1 Resources

A resource has a name, a kind and a state. A **flow resource** has a rate and a calendar. Its kind gives the unit of its rate: `TensorOps` (MACs), `VectorOps` (lane operations), `ScalarOps` (instructions) or `Bandwidth` (bytes). An **occupancy resource** (kind `Slots`) has a size and a queue of waiters. Table 11 gives the resources of each element of a configuration and their names. An element without a rate has no resource. Such a bus or link is unlimited: it adds its latency and nothing else.

Table 11: The resources of the elements of a configuration.

Element	Resources	Kind
memory block	`<mem>.port`; with `wr_bytes` also the write port `<mem>.wport`. A bank i of a banked memory is the block `<mem>.b<i>`.	`Bandwidth`
crossbar (`xbar`) with a rate	per attachment an ingress port `<bus>.<att>.in` and an egress port `<bus>.<att>.out`; with `wr_bytes` also `.in.wr` and `.out.wr`	`Bandwidth`
shared bus (`bus`)	`<bus>`; with `wr_bytes` also `<bus>.wr`	`Bandwidth`
link with a rate	one resource per direction: `<link>.ab` (from its `from` bus to its `to` bus) and `<link>.ba`; with `wr_bytes` also `.ab.wr` and `.ba.wr`	`Bandwidth`
link with `inflight_bytes`	the pool `<link>.inflight`	`Slots`
Atlas	`<pe>.array`	`TensorOps`
Muon	`<pe>.vector`	`VectorOps`
Neutrino	`<node>.scalar`, `<node>.complete`	`ScalarOps`
profile hold	a resource that the profile names	`Slots`

A compute master (an Atlas, a Muon) on a rated crossbar gets no port, because it is never the end of a route that charges. A DMA engine has no resource of its own. Its beat and clock start the chain of beats of its transfers (Section 5.6). The attachment of a memory, a PE or the Neutrino on a bus has the short name of the element, for example `hbm` or `neutrino`. The attachment of a link on a bus of the node of the link has the name of the link. On a bus of another node it has the path of the link from that node, for example `root.gpc0.clsbus.sm0.smsbus_to_clsbus.out`.

5.2 Slots and clock ratios

A flow resource has a rate R in units per cycle of its own clock, a clock ratio k, and, for bytes, a beat b (the key `beat_bytes`, default 64). A slot moves one beat in one SM cycle. The capacity of the resource in slots per SM cycle is

$$C = \frac{R}{b} \times k \quad \text{for bytes,} \qquad C = k \quad \text{for work (one slot whose beat is the full rate).}$$

Table 12 gives examples from the shipped configurations.

Table 12: Capacities of some resources.

Resource	R	b	k	C
2p: SMEM bank port	128 B	64 B	1	2
2p: HBM port of a cluster	512 B	64 B	1	8
RTX PRO 6000: L2 slice, read port	64 B	32 B	7/8	1.75
RTX PRO 6000: L2 slice, write port	32 B	32 B	7/8	0.875
RTX PRO 6000: GDDR7 channel	2 B	32 B	51/10	0.31875
RTX PRO 6000: GPC link, one direction	276 B	32 B	1	8.625
an Atlas array, a Muon vector unit	any		1	1

Capacities and usages are exact fractions. A calendar holds them in fixed point, with 65536×720720 units per slot. This is exact for every power-of-two ratio of beats up to 2^{16} and every clock denominator up to 16. A needed usage rounds up and a capacity rounds down, so a calendar never holds more than its capacity.

A clock ratio below 1 removes whole slots. Each remaining slot still moves a whole beat in one SM cycle. The slots of a resource exist at fixed cycles, from the absolute cycle only (Figure 6).

Rule S5.1 (whole slots by cycle). SM cycle t of a resource with capacity C has $\lfloor (t+1)C \rfloor - \lfloor tC \rfloor$ slots. The cycles $[t_0, t_1)$ have $\lfloor t_1 C \rfloor - \lfloor t_0 C \rfloor$ slots.

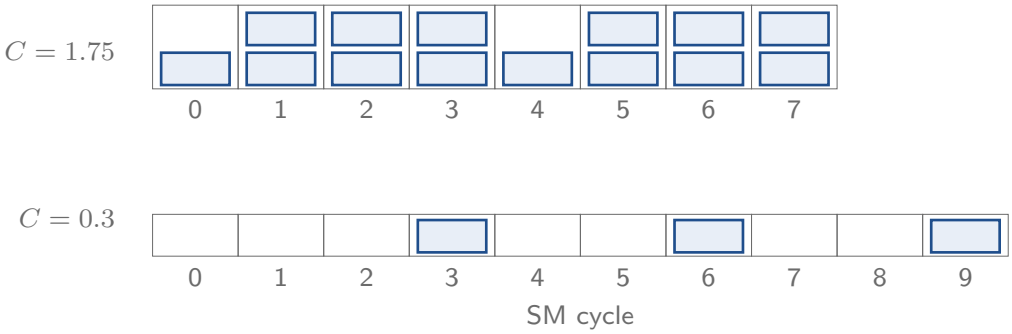


Figure 6: Slots by cycle for $C = 1.75$ and $C = 0.3$. Each square is one slot. The slots of a resource exist at fixed cycles.

A chunk holds a constant usage of u slots during its **window**: whole cycles of the resource. It moves the fraction u/C of the slots of each cycle of its window. In every cycle, the chunks on a resource share the slots of that cycle in proportion to their usages. A cycle without a slot moves nothing.

Rule S5.2 (window of a chunk). A chunk that moves n beats at a usage of u slots needs $N = \lceil nC/u \rceil$ slots of the resource ($N = n$ when $u = C$). Its window from t_0 ends at the first t_1 with $\lfloor t_1 C \rfloor - \lfloor t_0 C \rfloor \geq N$. With an integer capacity the window is $\lceil n/u \rceil$ cycles. A window is at least one cycle.

Example. An L2 slice read port of the RTX PRO 6000 configuration has $C = 1.75$. A chunk of 256 bytes in 32-byte beats has $n = 8$ beats. From cycle 454, at $u = 1$: $N = 14$, and $\lfloor 462 \times 1.75 \rfloor - \lfloor 454 \times 1.75 \rfloor = 808 - 794 = 14$, so the window is $[454, 462)$. At $u = 0.75$: $N = \lceil 18.67 \rceil = 19$ and the window is $[454, 465)$. At the full capacity: $N = 8$ and the window is $[454, 459)$.

A demand of work (MACs, lane operations, instructions) of amount A at utilization η has $\lceil A/(R\eta) \rceil$ beats of its own clock, one per slot (Section 7). With $k = 1$ its window is $\lceil A/(R\eta) \rceil$ SM cycles.

Rule S5.3 (whole cycles). Each charge holds whole cycles, at least one. Nothing carries over from one charge to the next: two messages of half a beat each take two cycles.

5.3 Calendars

The **calendar** of a flow resource is a sorted list of keys. A key is a cycle and the slots in use from that cycle to the next key. The resource plane uses these operations on a calendar:

- the earliest cycle $t' \geq t$ at which u slots are free during a window of length d; with a fractional capacity the length depends on the start, so the search repeats from each candidate until a window of its own length is free;
- the first cycle at or after t at which some slots are free;

- the slots that stay free during an interval;
- occupy and release u slots during an interval.

No reservation starts before the current cycle. So every 16 events the control plane removes the keys before the current cycle. A calendar then holds a few tens of keys.

Each flow resource also counts, for the statistics, the slot-cycles that it charged (split into read, write and work), the bytes, the culprit cycles and the delay cycles (Section 5.11).

5.4 Chunks

A task uses resources in chunks (glossary: transfer chunks and compute chunks). The chunk is the unit of arbitration. A task reserves its chunks one at a time, each in the earliest free window at or after its request. Two transfers on one hop therefore interleave chunk by chunk. This is an approximation of round-robin arbitration. A reservation of a whole transfer at once would give strict FIFO order: the second transfer would wait for all of the first one.

The size of a chunk is a knob of fidelity. One chunk per transfer gives FIFO order and the fewest events. One chunk per beat gives arbitration per cycle and the most events. The default is one tile: one operand or output tile of a matmul, one tile of a copy.

Rule S5.4 (size of a transfer chunk). A transfer chunk at source address s and destination address d moves

$$n = \min(\mathit{tile}, \mathit{left}, \mathit{run}(s), \mathit{run}(d), \mathit{grid}_r(s), \mathit{grid}_w(d))$$

bytes. *tile* is the `tile_bytes` of the descriptor, or the `burst_bytes` of the engine when it has one. $\mathit{run}(a)$ is the number of bytes from a to the end of its memory block; in the linear range of a hash link, to the end of the hash granule. $\mathit{grid}(a)$ is the number of bytes from a to the next multiple of the smallest `block_bytes` on the route half.

All block sizes and hash granules are powers of two. So every boundary of a larger grid is also a boundary of the smallest grid, and one grid per route half is exact. A chunk never crosses the boundary of a block, of a slice of a hash link, or of a transaction of a bus, link or port.

5.4.1 Issue of transfer chunks

A DMA engine moves at most one beat per cycle of its clock: its beat rate is $r = \mathit{beat} \times k$ bytes per SM cycle. It issues the chunks of a transfer back to back at this rate:

1. Chunk 0 issues at the issue of the task. Chunk $i + 1$ issues $\lceil n_i / r \rceil$ cycles after chunk i.
2. Each chunk reserves its hops at its own issue, with the earliest cycle $\max(\mathit{issue}, \mathit{task\ issue} + \mathit{fill}) + L$. L is the latency of the engine. The **pipeline fill** is the latency of the source block (Section 5.9).
3. With `outstanding` set, at most that many chunks are issued and not ended. A chunk over the cap issues when a chunk ends. Without the cap (the default), only the last chunk has an end event, at the latest end of all chunks.

The transfer pays its latencies once, as a pipeline fill, and the engine runs at its beat rate unless a hop is busy. No chunk is reserved before its own issue, so a transfer that starts later still interleaves with an earlier one, chunk by chunk. The key `inflight` limits the transfers that an engine serves at once.

A compute task has no beat rate. It requests `outstanding` chunks ahead (default 1), and each chunk end requests the next one.

5.5 Routing

The routing function finds the hops of a chunk. At the start of a run, the resource plane builds these tables:

1. the memory blocks, sorted by base address, for a binary search;
2. the hash links, by their linear range;
3. for each DMA engine and each memory block, the read half and the write half of the route.

A chunk then needs no search, no hash and no allocation. The block and the route half of a side change only when the address of the side leaves the run of its block.

Memory blocks by address. To locate an address a, the routing function first looks for a hash link whose linear range holds a. If one does, it rewrites a to the physical address and takes the block of that address; the run ends at the hash granule. Otherwise it takes the block of a and the run to the end of the block.

Hash links. A hash link (Section 3.3) has a linear range, a physical range, 1 to 32 index masks and one drop bit per mask. The rewrite of a linear address a is:

- slice index bit j is the parity of a & $index_j$;
- the offset in the slice is $a - base$ with the drop bits removed;
- the physical address is $to_base + index \times slice_bytes + offset$.

The hash granule is the lowest bit of any mask. A chunk inside one granule stays in one slice. The loader checks that the rewrite is a bijection from the linear range onto the physical range. Ownership, the Neutrino tables and the functional plane see only linear addresses. Only the routing function sees the physical blocks.

Example. `gpc-hbm-sliced.yaml` has `index: [0x200, 0x400]`: two masks of one bit, so four slices in a stripe of 512 bytes, and the drop bits are 9 and 10. The RTX PRO 6000 configuration has six masks of many bits each (the measured hash of the L2 slices). Bit 8 is the lowest bit of any mask, so the granule is 256 bytes. The linear addresses `0x000`, `0x100`, `0x200` and `0x300` go to the slices 0, 1, 0 and 1: bit 8 is in mask 0 only, and bit 9 is in no mask.

Routes over the fabric. The buses are the vertices of a graph and the links are its edges. A route is the path with the fewest links. Two different paths with the fewest links between two buses are a load error. Between two sets of attachments (a memory on two buses, a master on two buses), the route is the pair with the fewest links; on a tie, the first pair in the order of the configuration. Along a route:

- a rated crossbar gives two hops: the ingress port of the attachment where the route enters (latency 0), and the egress port of the attachment where it leaves (the latency of the bus);
- a shared bus gives its one resource, with the latency of the bus;
- a link gives the resource of its direction, with the latency of the link;
- a hop without a rate has no resource. Its latency adds to the rated hop before it, or to the lead of the route if no rated hop comes before;
- a bus or link with `wr_bytes` has a write channel. The route then has a second list of hops for writes. The latencies are the same.

Route halves. A transfer from block S to block D on engine E has the hops of two **route halves**:

- the read half: the read port of S (with the lead of the route from S to E as its latency), the rated hops from S to E, and the extra links of E;
- the write half: the rated hops from E to D on the write channels, and the write port of D.

The lead of the write route adds to the latency of the last hop of the read half. The same halves serve every chunk of every transfer between S, E and D.

Other routes. A load or store of a program takes the route between the data attachments of the Neutrino and the block (Section 5.10). A Muon task charges the rated hops of the routes from the block of its source and to the block of its destination as demands of its compute chunk (Section 6.3). An Atlas charges only the SMEM ports (Section 6.2). A message takes the control route between two Neutrinos (Section 4.7).

5.6 Charging a transfer chunk

A transfer chunk moves bytes over the hops of its route. Seven rules, the data-path rules, give its windows. Section 13.1 applies them to one chunk.

Rule S5.5 (route order). The control plane reserves the hops of a transfer chunk in the order of the data. Hop 0 takes its window at or after the earliest cycle of the chunk. Hop $i+1$ takes its window at or after the start of hop i plus the latency of hop i. The chunk ends at the latest end of its windows plus the latency of its last hop.

Rule S5.6 (beats). The beat of a chunk on hop i is $b_i = \min(b_{i-1}, beat_i)$. b_{-1} is the beat of the master. A beat splits at a narrower hop and never merges again. A narrow beat takes a whole slot.

Rule S5.7 (narrowing). A chunk comes into hop i at the rate r_{i-1} that it got on hop $i-1$; r_{-1} is the beat rate of the master. On hop i the chunk needs r_{i-1}/b_i slots, at most the capacity. When it holds u_i slots, its rate after hop i is $r_i = u_i \times b_i$. A slower hop downstream does not slow the hops upstream.

This **narrowing** carries the rate of the slowest hop so far to every later hop of the chunk: data cannot arrive faster than the slowest hop upstream.

Rule S5.8 (sharing). Hop i offers two windows. Window A is the earliest window in which the need of the chunk is free throughout. Window B starts at the first cycle at which some slots are free and holds the slots that stay free over it. The chunk takes the window that ends first; on a tie, window A. The usage is constant over the window. A reservation never changes after it is made.

Rule S5.9 (waiting between hops). Each hop keeps the window that Rule S5.5 gives it. When a later hop is busy, the data waits between the hops. Only the in-flight pools (Rule S5.11) limit this wait.

Rule S5.10 (bursts). With `burst_bytes` on the engine, each burst is a chunk under these rules. Each hop frees after its own window, so bursts of different transfers interleave on each hop.

Rule S5.11 (data in flight). A data link can declare `inflight_bytes`: an **in-flight pool** for the data that enters the node of the link through the link. A chunk whose data crosses the

link into that node holds its bytes in the pool for a time. The hold starts when the request of the chunk crosses the link, and it ends at the end of the window of the chunk on the link. The request crosses at $\max(issue, e - L_{src} - L_{req})$: e is the earliest cycle of the first hop, L_{src} the latency of the source block, and L_{req} the latency of the route from the link to the source block. On a link without a rate, the hold ends with the window of the first rated hop after the link. A chunk whose bytes do not fit waits: its earliest cycle moves later until the pool has room over the whole hold.

Note. Data that leaves the node, and messages, do not use a pool. A write into the node counts with $L_{src} = 0$. With a pool of N bytes and holds of H cycles, the link carries at most N/H bytes per cycle (Little's law). So a longer latency gives less bandwidth, as on a device that limits the data in flight per cluster. The shipped configurations do not use pools.

A message is not a transfer chunk. On each hop it holds one slot, or the full capacity of a hop with less than one slot. Its beat is the smaller of the beat of the sender and the beat of the hop. The hops before it do not narrow it (Section 4.7).

The statistics count slot-cycles. The utilization of a hop is its slot-cycles over its capacity. Its byte utilization (bytes over the rate) is lower where the beats of the chunks are narrower than the beat of the hop.

5.7 Charging a compute chunk

A compute chunk (an Atlas tile, a Muon task) uses its unit and its memory ports at the same time. The resource plane **co-charges** its demands from one common start:

1. It adds the demands of the chunk on the same resource. They must have the same utilization.
2. It computes the **own duration** of each demand: the window of its beats (or of its work) at the full capacity of its resource.
3. D is the longest own duration. It sets the duration of the chunk.
4. A data demand holds $beats/D$ slots (at most the capacity) for D cycles. So a tile that is bound by its unit leaves free slots on the port for a transfer beside it, and a tile that is bound by the port takes all the slots.
5. A work demand holds the full capacity of its unit for its own duration. (The setting `cocharge: longest` charges it for D.)
6. The start t is the earliest cycle at or after the earliest cycle of the chunk at which every demand has its usage free for its length. The resource plane finds it as a fixed point: t only moves later.
7. The chunk ends at t plus the longest charged length.

Rule S5.12 (start on the free slots). When the full usage delays the start of a compute chunk, the chunk can start at its earliest cycle on the slots that stay free on each data resource. It then holds $u' \le u$ slots on each port, and its duration D' is the longest of the own durations of its work demands and the windows of its data demands at u'. Each port then holds $beats/D'$ slots for D'. The chunk takes this option only when it ends strictly earlier.

Note. A real port arbitrates beats and lets a chunk go faster when another flow ends. A chunk with a constant usage cannot. So the option of Rule S5.12 gives a time between the full usage and an exact sharing of the port.

5.8 Occupancy resources

A profile can list holds: an occupancy resource and an amount. A task takes its holds at issue and gives them back at completion. A hold that does not fit puts the PE into the FIFO of the resource. Units that come back wake the waiters in FIFO order. A bound on outstanding requests, for example a number of transfers in flight, is a hold. It is separate from the latency. The built-in profiles have no holds. The in-flight pools are occupancy resources with a calendar of bytes over time (Rule S5.11).

Note. The tables of a Neutrino are capacities of the control plane (Section 4.4). They are not occupancy resources of the resource plane.

5.9 Latency

Latency is a delayed use of resources. It is not a resource. A chunk with latency L reserves its demands at or after its request plus L. A load of 64 bytes takes its latency, then a reservation of 64 bytes on its hops that lasts a few cycles. Table 13 gives where each latency applies.

Table 13: Latencies in the model.

Latency	Where it applies
engine (`latency` of a DMA engine)	before each transfer chunk, after the pipeline fill
pipeline fill	once per transfer: the latency of the block of the first source byte (of its slice, on a cache hit)
hop (`latency` of a bus or link)	between the start of a hop and the earliest start of the next hop; after the window of the last hop
Muon	the `overhead` of the Muon plus the longer of two paths, before its chunk: the latency of the block that it reads plus the route latency in, and the route latency out (a write pays no block latency)
load of a program	the route to the block and the latency of the block, before the read port
store of a program	the lead of the route before the first hop and the hop latencies; no latency of the block (a write pays none)
message	the lead of the control route plus `msg_latency` before the first hop; the hop latencies between the stages
$c_{resolve}$	from the last dependency of a task to ready; from a done key to the wake of a local wait; no resource

Rule S5.13 (source latency). A transfer must pay the latency of its source block once, before its first chunk, as part of its pipeline fill. It pays no latency for its destination block: a write is posted.

The model charges only the data of a transfer. The read requests and the write acknowledgements move against the data. They use no resource, and a transfer pays the latency of its route once, in the direction of the data. A load of a program also pays the route to the block (Table 13). Document 2 leaves open whether the writes of Radiance 2 are posted (Section 7).

5.10 Memory access of a program

A load or a store of a program has one piece per memory block that its range touches, cut at the runs and grids of Rule S5.4. Each piece is a transfer chunk in the order of the data, with the

Neutrino as its master (its `beat_bytes`, default 64). A load reads the port of the block and then takes the route back to the Neutrino. A store takes the route to the block on the write channels and then the write port. Table 13 gives their latencies. The access ends with its last piece.

5.11 Culprits and delays

Each chunk has a culprit (the resource with the longest own duration). For a transfer chunk, the culprit is the first hop with the longest window: the narrowest hop, whose rate the later hops get. For a compute chunk it is the demand with the longest own duration; on a tie, the one with the latest own start.

A chunk that starts after its earliest cycle also has a **queue culprit**:

- for a compute chunk, the demand whose own earliest start was the latest; its delay is the start of the chunk minus its earliest cycle;
- for a transfer chunk, the hop with the largest start delay (its start minus the earliest start that the hop before allowed); on a tie, the later hop;
- for a chunk that waited for an in-flight pool at least as long as the largest hop delay, the pool.

A resource adds to its culprit cycles the length of each chunk for which it is the culprit (for a transfer chunk, the length of its window on the culprit hop). It adds to its delay cycles the delay of each chunk for which it is the queue culprit. The outputs rank the resources by these sums (Section 11).

Note. Each window rounds up to whole cycles. A hop after the narrowest hop can therefore get a window one cycle longer than the narrowest hop and become the culprit. In the GEMM of Section 13.2, a chunk of 256 bytes holds the full capacity of a GDDR7 channel for 25 cycles. Its narrowed share of the GPC link takes 26 cycles. So the GPC link is the culprit of most chunks.

6 PE models

A PE model is a profile function. Each opcode that a PE executes registers one function in a registry at the start:

$$profile(descriptor, node) \rightarrow (chunks, holds, fill).$$

The **profile** of a task is its chunks, the occupancy that it holds and its pipeline fill. A chunk has either **demands** (a compute chunk) or the two route halves of a transfer (a transfer chunk), and a latency. The node gives the PE, the memory blocks, the routing tables, the fabric and the decisions of the accumulator buffer. The opcodes `join` and `spawn` have no profile, because they have no PE.

Rule S6.1 (pure profile). A profile function must depend only on the descriptor and the hardware configuration. It must not read data values.

Work that depends on data content, for example sparsity or an early exit inside a PE, enters a profile in one of two ways: a statistical parameter in the configuration (for example an effective density), or a field of the descriptor that the program fills after a load. The control plane computes the profile of a task once, at the first try to issue it. Table 14 gives the built-in profiles.

Table 14: The built-in profiles.

Opcode	PE	Chunks	Demands of a chunk	Latency
`dma`	DMA engine	one per transfer chunk (Rule S5.4)	the hops of the read half and the write half	engine latency, after the fill
`atlas.matmul`	Atlas	one per output tile	the array (MACs) and the SMEM ports (operand and C bytes)	0
`muon.vector`	Muon	one	the vector unit (lane operations), the route hops and the ports (bytes in and out)	overhead and route

6.1 DMA engine

A DMA descriptor gives the source address, the destination address, the bytes and the tile. The profile uses these arguments only. The destination range of the descriptor has no effect on the time.

- The chunks follow Rule S5.4. The `burst_bytes` of the engine, when set, replaces the tile.
- Each chunk has the read half of its source block and the write half of its destination block, for this engine (Section 5.5).
- The rate of each chunk is the beat rate of the engine, $beat \times k$. Its latency is the `latency` of the engine.
- The pipeline fill is the latency of the block of the first source byte.

In Radiance 2 a tracked copy into the memory of another node is a pull by the DMA engine of the owner (Section 5.1). An untracked copy can write into the memory of another node (Section 5.4); it runs on the engine of the invoking node. The engine reads over its read half and writes over its write half. The model makes no other difference between a pull and a push.

The control plane checks Rule R4.2 against the destination range of the descriptor. Only the functional plane checks Rule R5.1. So a run with the Null backend accepts a destination range that does not hold the bytes that the task writes. The store records of the `kernels` program use this (Section 13.3.2).

6.2 Atlas

A matmul descriptor gives the addresses of A, B and C, their leading dimensions, and m, n and k. It also gives the element types of A, B and C, the flags `accumulate` and `keep`, and the output tile. The profile has one chunk per output tile of $t_m \times t_n$ elements over the full k. A chunk has these demands:

- The array: $q_m \times q_n \times k$ MACs. With `quantize: tile` (the default) q_m and q_n are the full tile, so a tile of 51 rows costs a tile of 128 rows. With `quantize: none` they are the exact shape.
- The rate class: the array has the largest rate of the configured element types, R_{max}. A tile runs at the rate R_s of its slower operand type, as the utilization $\eta \times R_s/R_{max}$, with η the configured `utilization`.
- Reads from SMEM: the A tile ($q_m \times k$ elements; zero after the first tile of a row with

`operand_reuse: row_panel`), the B tile ($k \times q_n$ elements), and the C tile when the task accumulates and the accumulator buffer does not hold it.
- Writes to SMEM: the C tile, unless the accumulator buffer keeps it, and on the first chunk the bytes of the tiles that the buffer evicts.
- The ports: with one SMEM port, the reads charge the port and the writes the write port (or the port). With a banked SMEM, each operand range charges the ports of the banks that it overlaps, in proportion to its bytes in each bank. The bytes move in the beat of the Atlas.

The element sizes are 2 bytes (bf16), 4 (f32), 1 (fp8), and for mxfp4 half a byte plus one byte of scale per 32 elements. The number of MACs does not depend on the type.

The accumulator buffer (`acc_buffer`, in bytes, per Atlas unit) keeps C tiles between the matmuls of one chain. The control plane decides at the profile of each task:

- a task whose destination the buffer holds reads C from the buffer, not from SMEM;
- a task with `keep` whose C fits stays in the buffer and does not write C to SMEM;
- the buffer evicts its least recently used tiles to make room; their bytes are a write to SMEM on the first chunk of the task.

The accumulator buffer stands for the tensor memory of document 2 (Section 5.2) until that memory is specified. Only C tiles stay in it: the profile reads the A and B tiles of each output tile from the SMEM. The profile charges only the ports of the SMEM, not the buses between the Atlas and the SMEM. It does not check that the operands are in the SMEM of the SM (Rule R5.3). It checks only the hashed ranges (Rule S6.2).

Rule S6.2 (no hashed operand). A compute PE must not address the linear range of a hash link. Such a task stops the run.

Document 2 has the same rule (Rule R3.6). The model needs it: the profile of a compute task charges one memory block per operand, and it does not route an operand through a hash link.

6.3 Muon

A vector descriptor gives the lane operations, the bytes in, the bytes out and, as an option, a source address. The profile has one compute chunk:

- the vector unit: the lane operations at the `utilization` of the Muon;
- the bytes in: a read from the block of the source address (of the destination, without one), on the rated hops of the route from that block to the Muon and on the read port of the block;
- the bytes out: a write to the block of the destination, on the rated hops of the route from the Muon and on the write port of the block;
- on a banked block, the bytes of each side spread over the banks of its range.

The latency of the chunk is the `overhead` of the Muon plus the longer of the two route latencies. All demands are co-charged (Section 5.7). The model does not interpret Muon instructions (Section 9).

7 Utilization

Utilization is the knob that connects the model of a unit to a measurement. This section gives its definition, where it comes from, how a measurement sets it, and its role in mixed mode.

7.1 Definition

Utilization is exclusive. A resource of rate R that serves a demand of amount A at utilization η is busy for

$$\frac{A}{R \times \eta}$$

cycles of its clock, and no other work can use the idle fraction. For example, a Muon that issues instructions at half of its peak rate has $\eta = 0.5$: it holds the vector unit twice as long for the same lane operations.

Rule S7.1 (exclusive utilization). A utilization must be in $(0, 1]$. A demand at utilization η must hold its resource for the time of A/η units at the full rate.

On a work resource, the window of the demand is $\lceil A/(R\eta) \rceil$ cycles of its clock (Section 5.2). A data demand of a compute chunk can also have a utilization: its number of beats then grows by the factor $1/\eta$. A transfer chunk has no utilization.

7.2 Sources

Table 15 gives where a utilization comes from. Two demands of one chunk on the same resource must have the same utilization.

Table 15: Sources of a utilization.

Source	Applies to	How
`utilization` of an Atlas	each matmul tile	times the ratio of its rate class, R_s/R_{max} (Section 6.2)
`utilization` of a Muon	each vector task	directly
a profile function	one demand	the utilization field of the demand
a program	one task	the program divides the amount by η; for example `simt_util` of the `kernels` program (Section 9.2)

A program that divides the amount gets the same window as a utilization in the profile. It can then choose a value per task, for example per kernel variant, with no change to the configuration.

7.3 Calibration

A measurement of c cycles for a chunk of amount A on a unit of rate R gives

$$\eta = \frac{A}{R \times c}.$$

The model has no table of latencies for tasks. A task runs from its issue through the windows of its chunks to its completion. Its duration follows from its chunks and is never an input.

Rule S7.2 (no explicit duration). The model must not give a task an explicit duration. A measured efficiency of a unit must enter the model as a utilization of a demand.

A profile function decides how it keeps its values: the key (for example the fields of a descriptor that set the efficiency), the storage and when it computes them again. There is no global policy. The bandwidth of a tile changes with its shape and is computed at each profile.

7.4 Mixed mode

In mixed mode, one subtree of the device runs in an RTL simulation (or on an FPGA) and the rest runs in r2-sim. Utilization is the knob that carries the time of the RTL into the model:

1. The RTL subtree is the backend of the functional plane for its subtree. It supplies data values, as the Numeric backend does.
2. For a sample of the chunks of a type, the RTL also reports its cycles c. The profile function computes $\eta = A/(Rc)$ for that type.
3. Later chunks of the type use the updated utilization. The control plane keeps the order of the work, so the RTL is not on the critical path of the event loop.

A co-simulation in which the RTL gives the time of each of its chunks needs the two simulators in lock step. That is a different method. It is not part of the decoupling of the planes.

Note. The code does not have the RTL backend or the update of utilizations from sampled runs. In the current code a utilization is constant during a run: it comes from the configuration and from the program. The calibration of the SIMT units against a card (Section 9.3) uses the same knob, with a GPU in place of the RTL.

8 Cache model

A device can have a memory that caches a DRAM, for example the L2 of a GPU in front of its DRAM channels. The cache model decides, chunk by chunk, whether the bytes of a DMA come from the cache or from the DRAM, and it adds the traffic of evictions. The model is minimal (Section 8.5). The shipped Radiance 2 configurations have no cache. The RTX PRO 6000 configuration has one on its L2 (Section 13.3).

8.1 Configuration

A scratch memory becomes a cache with the key `cache: {of: <dram memory>, line_bytes: L}` (Section 10.4).

- The banks of the scratch memory are the **slices**. The banks of the DRAM are its channels.
- The size L of a **line** is a power of two (default 256 bytes). The banks of the scratch memory must have equal sizes that are multiples of L.
- A slice holds its bank size over L lines.
- A DMA engine of the owner of the slices must reach a slice and a channel. Its route halves carry the write-backs.

8.2 State

The cache keeps this state (Figure 7):

- The line of a linear address a is $\lfloor a/L \rfloor$.
- The channel of a line is the bank of the DRAM that holds its address. For a DRAM behind a hash link, this is the bank after the rewrite.
- The slice of a line is its channel modulo the number of slices.
- Each slice keeps its resident lines, a dirty bit per line, and the order of their last use.

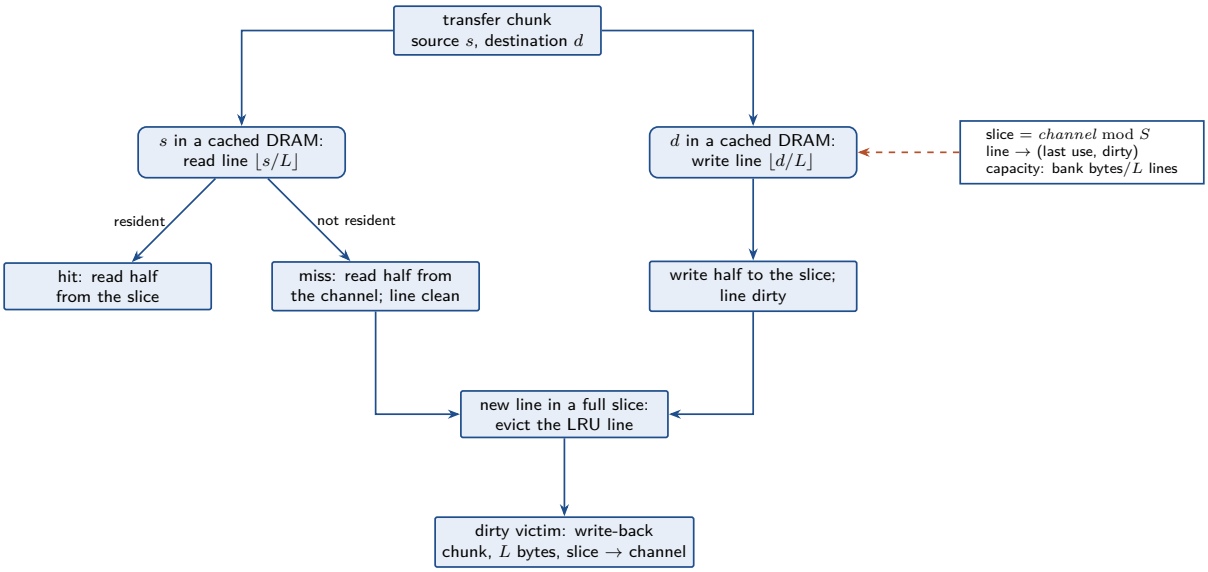


Figure 7: The bookkeeping of the cache for one transfer chunk.

8.3 Bookkeeping

The bookkeeping runs when the control plane builds the profile of a DMA, before the issue of the DMA (Section 4.5). It does these steps for each chunk:

1. It cuts the chunk at the line boundaries of each side that is in a cached DRAM. A chunk then touches at most one line on such a side.
2. Source in a cached DRAM, the line is resident (read hit): the read half of the chunk becomes the read half from the slice. For the first chunk of the task, the pipeline fill becomes the latency of the slice.
3. Source in a cached DRAM, the line is not resident (read miss): the read half stays the read half from the channel. The line becomes resident and clean.
4. Destination in a cached DRAM: the write half becomes the write half to the slice. The line becomes resident and dirty. A line that is not resident is allocated with no read from the channel.
5. An access makes its line the most recently used line of its slice.
6. A new line in a full slice evicts the least recently used line of the slice. A dirty victim adds a **write-back chunk** to the task, right after the chunk that evicts: L bytes from the slice to the channel of the victim, over the route halves of the write-back engine.

A write-back chunk has latency 0 and the beat and rate of the engine of the task. It is a transfer chunk like the others (Section 5.6).

Example. The `kernels` program on the RTX PRO 6000 configuration with `cache = "sim"`: two kernels on two SMs, a layer norm that reads `x` and writes `z`, then an add that reads `z` and `x` and writes `z`; each tensor has 64 KiB, so 256 lines. The device flushes the caches before each kernel. The counters of the layer norm: 256 read misses (the lines of `x`) and 256 write allocations (`z`). The counters of the add: 512 read misses, because the flush removed every line, and 256 write hits, because the reads of `z` made its lines resident.

8.4 Flush

A program call `cache_flush()` empties every cache. It moves no data and costs no cycle. Dirty lines go away without a write-back. The call stands for the flush of a measurement tool between two kernels, outside the measured time. The cache records its counters at each flush, so the statistics give the counters of each interval.

Table 16: Counters of a cache.

Counter	Meaning
`read_hits`, `read_misses`	reads of a resident line, of a line that is not resident
`write_hits`, `write_allocs`	writes to a resident line, to a line that is not resident (allocated dirty)
`allocs`	lines made resident (read misses and write allocations)
`clean_evictions`, `dirty_evictions`	evicted lines by state
`writeback_bytes`	bytes of the write-back chunks
`peak_lines`	the most resident lines at one time since the start or the last flush

8.5 Limits of the model

- The bookkeeping follows the order in which the control plane builds the profiles of the tasks. It does not follow the order in which their bytes move.
- A read that hits a line that is still in transfer from the channel reads the slice at once (hit under miss).
- A read miss moves the data from the channel to the engine. It does not write the line into the slice and does not charge the port of the slice.
- The cache has no sectors, no sets and no other replacement than least recently used. The channel of a line selects its slice; the slices have no hash of their own.
- A write-back is a chunk of the task that evicts. There is no write-back buffer.
- Only DMA tasks use the cache. Loads and stores of programs, and the operands of compute PEs, do not.

9 SIMT modeling

The Muon is the SIMT core of an SM. r2-sim does not interpret its instructions. A Muon task is opaque: a number of lane operations, the bytes that it reads and the bytes that it writes (Section 6.3). Its time on the vector unit is its lane operations over $R \times \eta$. This section gives what the count stands for, how a program sets the utilization, and the method that calibrates and checks it.

9.1 What a lane operation counts

The plan of a kernel counts its arithmetic per element: one lane operation per add, multiply or fused multiply-add, and 8 per operation of the special function unit (an exponential, a hyperbolic tangent, a reciprocal square root). The factor 8 is the ratio of the two issue rates on the measured card: 128 lanes against 16 operations of the special function unit per cycle. A real SIMT core issues more instructions per element: address arithmetic, type conversions, shuffles and barriers.

It also stalls on dependent reductions and on barriers. The model has no warps, no occupancy and no register pressure. One utilization per **kernel variant** carries the difference.

9.2 Utilization per kernel variant

The program sets the utilization of each task. The Muon of the configuration keeps its default utilization of 1. The `kernels` program has two parameters per kernel:

- `simt_util`: the utilization of the Muon tasks of the row operations (softmax, layer norm, elementwise);
- `softmax_util`: the utilization of the Muon tasks of an attention kernel (the online softmax and the final normalization). The tensor units of attention stay at utilization 1.

A value must be in $(0, 1]$; the default is 1. The program charges $\lceil ops/\eta \rceil$ lane operations for a task of *ops* lane operations. This is the exclusive utilization of Rule S7.1. The report of each kernel keeps the lane operations of the work (`lane_ops`), the charged ones (`lane_ops_charged`) and the value (`simt_util`).

Example. A layer norm in pieces of 2048 elements with 8 lane operations per element and `simt_util = 0.25` gives Muon tasks of $2048 \times 8/0.25 = 65536$ lane operations. On a Muon of 128 lane operations per cycle each task holds the vector unit for 512 cycles. The assembly listing shows `vec ... ops=65536` (Section 11.6).

9.3 Calibration and validation

The utilization of a SIMT kernel variant is a fitted value. These rules keep the fit from hiding errors of the model.

Rule S9.1 (only SIMT is fitted). Only SIMT work can have a utilization below 1. The tensor units, the copy engines, the memories and the links must use rates that isolated measurements give. The time of a whole kernel must not set them.

Rule S9.2 (one value per variant). There must be one utilization per kernel variant: the kernel function and its integer template arguments. The library of the device selects the variant from the shape, so the plan of a test reads the variant from the name of the kernel that the device ran.

Rule S9.3 (calibration with the model). A fitted value must come from one **calibration shape** of its variant. The calibration must solve "model cycles = measured cycles" with the model itself, so that the memory path of the model is part of the calibration. A validation test must not use a calibration shape.

Rule S9.4 (held-out shapes). Every other shape of the variant must use the value unchanged. These shapes are the **validation shapes**.

Rule S9.5 (report gaps). When a value does not carry to the validation shapes of its variant (a difference of more than 15 %), the difference must be reported as a **model gap**. A term that depends on the shape must enter the model only with a structural reason and with its own validation shape.

Rule S9.6 (recalibrate). A change to the structure of the model must repeat every calibration.

The calibration of one variant has these steps:

1. Run the model on the calibration shape at two values of η.
2. Find the η at which the model cycles equal the measured cycles, with the secant method on the model runs, to within 0.5 %.
3. Run the model on each validation shape of the variant with that η, and report the ratio of model cycles to measured cycles.

9.4 Kernel families

Some variants form a family: their code is the same and only an integer template argument n changes, for example the elements per thread of a register softmax. Each member is its own variant, so a value per member would have no validation shape. If the code of the family gives a structural form, the method treats the family as one variant with a form of two parameters:

$$SIMT\ cycles\ per\ CTA = T_0 + T_1 n.$$

T_0 is the part that does not depend on n (for a softmax, the two reductions over the CTA). $T_1 n$ is the work per element.

1. Calibrate two members, n_1 and n_2, as two calibration shapes.
2. Convert each η to SIMT cycles per CTA: $ops/(lanes \times \eta)$.
3. Take T_0 and T_1 from the line through the two points.
4. Give every other member $\eta = ops/(lanes \times (T_0 + T_1 n))$. Its shapes are validation shapes.

A variant with only two measured shapes cannot both fit and check a form of two parameters. Its gap stays a recorded model gap until a third shape is measured.

10 Configuration

A configuration describes the hardware of one device. It is one tree of YAML files with includes. The program parameters (TOML) and the trace configuration (Section 11.5) are separate files. The code fixes no part of the shape of the device.

10.1 Files

- A file has two sections: `global` and `device`. The device section is the root node.
- Every node has the same sections: `fabric`, `memory`, `compute`, `neutrino` and `children`, and the key `fidelity`.
- A child entry has a `name` and either its sections inline or `config: FILE`, a file relative to the file that includes it. The other keys of the entry override the file: mappings merge key by key, and other values replace the value of the file.
- An included file has no `name`: the name comes from the entry of the parent.
- A file can also be `base: FILE` with `set: [PATH=VALUE, ...]`: the base configuration with overrides.

The loader expands the includes, applies the overrides, resolves the names, and then builds the Neutrinos, the resources, the routes and the PEs. Listing 1 shows the file of one SM.

Listing 1: The SM of the 2p configuration (`sm.yaml`, comments removed).

```
fabric:
  - {name: smsbus, topology: xbar}
```

```
  - {name: smcbus, topology: xbar, control: true}
  - {name: lbus, topology: xbar}
  - {name: smsbus_to_clsbus, topology: link, from: $this.smsbus,
     to: $parent.clsbus, bw_bytes: 128, wr_bytes: 128, latency: 10}
  - {name: smcbus_to_clcbus, topology: link, from: $this.smcbus,
     to: $parent.clcbus, bw_bytes: 128, latency: 10, control: true}
memory:
  - {name: smem, type: scratch, addr: [[0x0, 0x100000]], banks: 2,
     bw_bytes: 128, wr_bytes: 128, latency: 10,
     slave: [$this.lbus, $this.smsbus]}
  - {name: sync, type: sync, addr: [[0x1F0000000, 0x1F1000000]]}
compute:
  - {name: atlas, type: tensor, rate: {bf16: 8192, fp8: 16384, mxfp4: 32768},
     quantize: tile, operand_reuse: none, acc_buffer: 1048576,
     master: [$this.lbus]}
  - {name: muon, type: vector, rate: {lane_op: 256}, overhead: 50,
     master: [$this.lbus, $this.smsbus]}
  - {name: dma, type: dma, latency: 100, inflight: 4, beat_bytes: 256,
     master: [$this.lbus, $this.smsbus]}
neutrino: {scoreboard: 128, pending: 256, execution: 256, versions: 256,
           inbound_refs: 128, inbound_invokes: 64, outbound_deps: 128,
           outbound_invokes: 32, c_invoke: 8,
           master: [$this.smcbus, $this.lbus, $this.smsbus]}
```

10.2 Names

Every element has a dotted name from the root, for example `root.gpc1.sm2.atlas` or `root.gpc0.clsbus`. In a reference to a bus, `$this` names the node that the file defines and `$parent` its parent. The shipped configurations name the buses by level and role: `sbus` and `cbus` at the device, `clsbus` and `clcbus` in a cluster, `smsbus` and `smcbus` in an SM, and `lbus` for the local scratch (`s` for data, `c` for control). A link has a free name, by convention `<from>_to_<to>`. Bank i of a memory is `<mem>.b<i>`.

10.3 Addressing

The key `global.address` gives the fields of an address: one field per level below the root (a name and a number of bits) and `offset_bits`. An address is [field 1] ... [field n] [offset]. The field of a child is its index among the children of its parent plus one. The value 0 at a level is the own space of the parent. The scratch, DRAM and sync ranges of a node are offsets in its own space, and the node owns them. A range whose size is not a power of two aligned to its base becomes the fewest aligned address sets (Section 2). The key `neutrino_granule` gives the granule.

Example. The 2p configuration has `offset_bits: 33` and the fields `gpc` (2 bits) and `sm` (3 bits). The own space of cluster g starts at $(g+1) \ll 36$. The own space of SM s of cluster g starts at $((g+1) \ll 36) \mid ((s+1) \ll 33)$.

10.4 Keys

Table 17 gives the keys of each element and the value of a key that a file does not set. Where document 2 names the parameter of a key in other words, the table gives that name in italics (Table 3). A rate (`bw_bytes`, `wr_bytes`, a compute `rate`) is per cycle of the clock of its element.

A latency or a cost is in SM cycles. A size is in bytes. A number is a YAML number, or a string with a fraction (`"7/8"`) or a unit (`"512 KiB"`, `"0x1000"`). The keys marked *model* are settings of the simulation. They are not parameters of Radiance 2.

Table 17: The keys of the configuration.

Key	Meaning	Absent
Global (`global`)		
`neutrino_granule`	The granule: the unit of the addresses of a Neutrino (Section 2.3). 512 in the 2p configurations, 128 in `rtx6000.yaml` and `rtx6000-half.yaml`.	required
`address.fields`	One field per level below the root, from the top: a `name` and `bits`.	no field
`address.offset_bits`	The width of the offset.	required
`clock_ghz`	The *SM clock*. Only the reports use it (Section 2.1).	no seconds
`msg_bytes`	The *message beat*.	64
`hop_queue`	The *stage depth*: the messages of one protocol class that a stage holds.	1
`flow`	The *flow control* of the invoke channel: `readyvalid` or `ticket`.	`readyvalid`
`msg_latency`	Model: cycles added to the lead of every protocol message except Start and Return.	0
`cocharge`	Model: `own` or `longest` (Section 5.7).	`own`
`livelock_cycles`	Model: the idle cycles with resends after which the run stops with a deadlock (Section 4.9).	10 × the largest `t_retry` plus the longest route latency
Node		
`name`, `children`	The name of the node; its child nodes, in order.	required; none
`fidelity`	Model: `null` or `numeric` (Section 10.4).	from the parent; `null` at the root
Bus (an entry of `fabric`)		
`topology`	`xbar` (a crossbar) or `bus` (a shared bus). The value `ring` is not implemented.	`xbar`
`control`	True for a control bus.	false
`bw_bytes`	The *bandwidth* of each port (`xbar`) or of the bus (`bus`).	unlimited
`wr_bytes`	The *write bandwidth* of a write channel. Only on a data bus with `bw_bytes`.	one channel
`beat_bytes`	The beat, the *transfer width* (Section 5.2).	64
`clock`	The *clock ratio*, an integer or `"p/q"`.	1
`block_bytes`	The *transaction size*, a power of two.	no cut
`latency`	The hop latency.	0
Link (an entry of `fabric` with `topology: link`)		
`from`, `to`	The two buses.	required
`control`, `bw_bytes`, `wr_bytes`, `beat_bytes`, `clock`, `block_bytes`, `latency`	As for a bus, for each direction.	as for a bus
`inflight_bytes`	Model: the in-flight pool for the data into the node of the link (Rule S5.11).	no pool

Key	Meaning	Absent
`hash.range`, `hash.to_range`	The linear range and the physical range, as offsets in the own space of the node.	no hash link
`hash.index`, `hash.drop`	The index masks, one per bit of the bank index; the drop bits.	drop: the mask bits, when each mask has one bit
Memory (an entry of `memory`)		
`type`	`scratch`, `dram`, `sync` or `cache`.	required
`addr`	The range, as an offset range in the own space of the node. A `cache`: global ranges.	required
`slave`; `master`	The buses that the memory is a slave on; the bus that a `cache` is a master on.	required
`banks`; `bank.b<i>`	The number of equal contiguous banks; the keys below for bank i.	1; none
`bw_bytes`; `wr_bytes`	The bandwidth of the read port and of the write port, per bank.	required; one port
`beat_bytes`, `clock`, `block_bytes`	As for a bus.	64, 1, no cut
`latency`	The latency of each memory block.	0
`cache.of`, `cache.line_bytes`	The DRAM that a scratch memory caches, and the line size (Section 8).	no cache; 256
Every PE (an entry of `compute`)		
`type`; `master`	`tensor` (an Atlas), `vector` (a Muon) or `dma` (a DMA engine); the buses that the PE is a master on.	required
`inflight`	The *tasks in flight*; for a DMA engine, the *transfers in flight*.	1
`outstanding`	A DMA engine: the *outstanding bursts*, a limit on the chunks that are issued and not ended. Another PE: the chunks requested ahead.	no limit; 1
`beat_bytes`, `clock`	The *transfer width* of the PE as a master; its *clock ratio*.	64, 1
`utilization`	Model: the exclusive efficiency, in $(0, 1]$ (Section 7).	1
Atlas (`type: tensor`)		
`rate.bf16`, `rate.f32`, `rate.fp8`, `rate.mxfp4`	The *MAC rate* for each data type.	at least one
`quantize`, `operand_reuse`	Model: `tile` or `none`; `none` or `row_panel` (Section 6.2).	`tile`; `none`
`acc_buffer`	The accumulator buffer, which stands for the *tensor memory* (Section 6.2).	0
Muon (`type: vector`)		
`rate.lane_op`	The *rate* of lane operations.	required
`overhead`	The *task overhead*.	0
DMA engine (`type: dma`)		
`latency`	The *engine latency*.	0
`burst_bytes`	The *burst size*: the chunk size of every transfer.	the tile of the descriptor
Neutrino (`neutrino`)		

Key	Meaning	Absent
`master`	One control bus and the data buses.	the first control bus, every data bus
`beat_bytes`	The *transfer width* of the loads and stores.	64
`scalar_rate`, `complete_rate`	The rates of the scalar resource and of the completion port (Section 4.3).	1
`scoreboard`, `pending`, `execution`, `versions`, `inbound_refs`, `inbound_invokes`, `outbound_deps`, `outbound_invokes`, `t_retry`	The capacities of the tables and the retry time (Section 11).	the values of document 1
`issue_headroom`	The issue headroom.	the number of PEs of the node
`c_*`	The costs of the instructions (Section 4.3).	the values of document 1

Fabric. A memory is a `slave` on one or more buses. A compute element is a `master` on one or more buses. By default the Neutrino masters the first control bus of its node (for messages) and every data bus (for loads and stores). A link joins two buses, usually `$this.x` and `$parent.y`. It is full duplex. Every node except the root needs a data link and a control link to buses of its parent. Data links join data buses and control links join control buses, so data and messages never share a resource. A memory of `type: cache`, a slave on one bus and a master on another, is a link through its port. It has the rate and the latency of its port and no hits, misses or capacity. The cache model with hits and misses is the cache of a scratch memory (Section 8). No shipped configuration has a memory of `type: cache`.

Fidelity. A node can declare its **fidelity**: `null` or `numeric`. A child inherits the fidelity of its parent; the root has `null`. The memory blocks of a subtree with fidelity null keep no data in the functional plane (Section 3.1).

10.5 Overrides

An override is `PATH=VALUE`. The command line takes any number of them with `-set`; a base file takes them in `set`. PATH is the dotted name of an element followed by keys inside it, for example `root.gpc0.sm1.atlas.rate.bf16` or `global.msg_latency`. A `*` matches any characters inside one component, for example `root.gpc*.sm*.smem.bw_bytes=512`. `[i]` selects an item of a list. `<mem>.b<i>.<key>` and `<mem>.b*.<key>` set a key of one bank or of every bank. VALUE is YAML. An override that matches nothing is an error.

Rule S10.1 (separate configurations). A hardware file must not hold program parameters or trace settings. A `-set` key that starts with `trace.` sets the trace configuration only.

Five flags of the command line also change the model: `-acc-buffer` (the accumulator of every Atlas), `-operand-reuse`, `-quantize`, `-cocharge` and `-flow`. The raw dump records them with the overrides.

10.6 Checks

The loader stops with an error on these conditions:

- two different routes with the fewest links between two buses;
- a child without a data link and a control link to its parent;
- a node with more than one sync-only range (`type: sync`, Rule R3.1);
- an owned range that holds address 0, the null address (Rule N2.1);
- `wr_bytes` on a control bus or link, or on one without `bw_bytes`;
- a hash link whose rewrite is not a bijection, a memory that overlaps its linear range, or memory blocks that do not cover its physical range;
- `inflight_bytes` on a control link, or on a link without an end in its own node;
- an execution table that does not exceed the issue headroom;
- a cache whose memory is not a DRAM, or whose slices are not equal multiples of the line.

It gives a warning when the beat of a master exceeds the rate of every bus that it masters. It also gives a warning when the outstanding bursts of a DMA engine do not cover its latency times its rate.

10.7 Shipped configurations

The directory `r2-sim/configs` holds the shipped configurations (Table 18). They are examples of document 2 (Section 2.5): they give values for studies and for the comparison with a GPU, and they do not fix Radiance 2.

Table 18: The shipped configuration files.

Files	Content
`2p.yaml`, `gpc.yaml`, `sm.yaml`	The 2p family: 2 clusters of 4 SMs, a cluster DRAM per cluster.
`2p-4sm.yaml`, `2p-16sm.yaml`	1 cluster of 4 SMs; 4 clusters of 4 SMs.
`2p-cmem64.yaml`, `2p-cmem-large.yaml`	A cluster memory of 64 MiB or 1.5 GiB per cluster (`gpc-cmem.yaml`).
`2p-smem2m.yaml`, `2p-smem512.yaml`	An SMEM of 2 MiB per SM, at 128 or 256 B/cycle per bank port.
`2p-hbm-sliced.yaml`	A cluster DRAM as 4 banks behind a hash link (`gpc-hbm-sliced.yaml`).
`2p-smallrefs.yaml`	Smaller Neutrino tables.
`rtx6000.yaml`, `rtx6000-gpc.yaml`, `rtx6000-sm.yaml`	The RTX PRO 6000 reference: 12 clusters of 16 SMs, with the L2 and the GDDR7 memory at the device (Section 13.3).
`rtx6000-half.yaml`	Half of the RTX PRO 6000 reference (its MIG partition 2g.48gb): 6 clusters, half of the L2 and half of the GDDR7 channels.

The RTX PRO 6000 reference gives the GPU the structure of Radiance 2. Every node has a Neutrino, and the device has large tables. The L2 is a scratch memory with a cache model. The GPU has no Neutrino, and its SMs store data directly. The programs for this reference emulate these stores with DMA tasks (Section 13.3.2).

11 Outputs

A run of `r2sim run` prints a summary and can write a statistics file, phase statistics, a trace and assembly listings. This section gives what each output holds and how the simulator computes it. Table 19 gives the options of the command line.

Table 19: Options of `r2sim run`.

Option	Effect
`-config FILE`	the hardware configuration (Section 10)
`-program P`	the program: `adamoe`, `prefill`, `forward`, `micro` or `kernels`
`-params FILE`, `-param k=v`	the program parameters (TOML)
`-trace FILE`, `-synthetic SEED`	the records of the Trace backend (Section 3.1)
`-set PATH=VALUE`	an override of the configuration (Section 10.5)
`-acc-buffer`, `-operand-reuse`, `-quantize`, `-cocharge`, `-flow`	settings of the model
`-stats OUT`	the statistics file
`-timeline`, `-phase-stats`	the task timeline, the phase statistics
`-trace-out DIR`	CSV files of the run
`-trace-raw DIR`	the raw dump, with `-trace-config FILE`, `-trace-zstd` and `-set trace.KEY=VALUE`
`-asm DIR`	the assembly listings
`-no-diagnostics`	no report of open pending entries, no late-consumer check

11.1 Summary

The summary gives the end cycle, the number of events and the wall time. It also gives the five resources with the largest share of culprit cycles and the three with the largest share of start delay. For example, the GEMM of Section 13.2 prints (shortened):

```
end 7857 cycles ... wall 0.45 s, 530 events
  culprit root.gpc0.clsbus_to_sbus.ba   24.3% of chunk cycles, utilization 0.031
  culprit root.gpc0.sm0.atlas0.array   15.8% of chunk cycles, utilization 0.521
  ...
  queued on root.gpc0.sm0.smsbus_to_clsbus.ba   97.1% of start delay
```

11.2 Statistics file

The option `-stats OUT` writes the run as JSON. Table 20 gives its main parts.

Two utilizations describe a resource over a run of T cycles:

$$\textit{utilization} = \frac{\textit{busy slot-cycles}}{T \times C}, \qquad \textit{byte utilization} = \frac{\textit{bytes}}{T \times R \times k}.$$

The byte utilization is lower than the utilization where narrow beats hold whole slots.

Table 20: Main parts of the statistics file.

Part	Content
run	end cycle, events, tasks, messages by kind and by beats, Await rejects and resends, stall cycles per Neutrino, warnings by kind, wall time
`resources`	per resource: kind, slots, rate, busy slot-cycles (and their read, write and work parts), bytes, utilization, byte utilization, culprit cycles and share, delay cycles and share; for an occupancy resource its size and peak; for a pool its mean occupancy
`culprits`, `delays`	the resources ranked by culprit cycles and by delay cycles
`stats.stages`	per message stage: the cycles in which its head could not move, and its peak occupancy
tables	the occupancy of every table at the end, the peaks of the scoreboard, version and invoke tables, the pending entries still open with the last writer of their range
`stats.tasks`	with `-timeline`: per task its invoke, ready, issue, complete and retire cycles and the interval and culprit of each chunk
`stats.loads`	per sync point: its cycle, range, the sequence number it waited for, and the ops emitted and applied at that time
`stats.caches`	per cache: its counters, and its counters at each flush
program	the report of the program: per kernel for `kernels`, the measured quantities for `micro`, the phases and steps for the forward-pass programs
fatal, unfinished	the first fatal error; the programs that did not finish, with what blocks them

11.3 Phase statistics

A program can record **marks** with `mark(name)`. With `-phase-stats`, the simulator keeps the totals of every resource (busy, culprit and delay cycles) at each mark. For each interval between two marks it reports three lists of five resources: the largest shares of culprit cycles, the largest shares of delay cycles, and the largest utilizations. A mark costs nothing and does not change the run. The `kernels` program marks the start and the end of each kernel.

11.4 Culprits and delays

The culprit cycles of a resource count where chunks spent their time with this resource as their limit. The delay cycles count where chunks waited before their start (Section 5.11). The share of a resource is its cycles over the sum of all resources. With `-trace-out`, the file `tree.json` sums the culprit cycles per node: each node reports its largest own culprit and the largest culprit of its subtree. With this file a reader can go down the tree to the resource that limits a phase.

11.5 Tracing

The control plane calls a tracer at fixed observation points (Table 21). A tracer has no return value, and the simulator reads nothing back from it.

Rule S11.1 (observation only). A tracer must not change a run. A traced run and an untraced run must end at the same cycle with the same events. With no tracer, each observation point must cost one branch.

Two tracers exist, and one export uses the first:

- The collector keeps the task timeline (`-timeline`), the interval log of every flow resource and the marks in memory, for the statistics file.

Table 21: Observation points.

Point	When
`task_invoke`	a task enters the tables of its owner, with the cycle of the call (and, forwarded, when it was queued and sent)
`task_ready`, `task_issue`, `task_complete`, `task_retire`	the steps of the task lifecycle
`chunk_grant`	the resource plane grants the interval of a chunk, with its culprit and queue culprit
`charge`	each charge of a calendar, per direction, with its owner
`route`	the bytes of a chunk, load or store over its route, unlimited links included
`message_free`	a message on a route without a rated hop
`dep_remote_open`, `dep_remote_done`	a remote dependency opens, is satisfied
`entry_open`, `entry_close`, `entries_clear`	an entry of a table (pending, version, inbound refs, inbound invokes, outbound deps, outbound invokes) changes
`program_start`, `program_return`	a program starts, returns
`mark`	a program records a mark
`finish`	the run ends, normally or on a fatal error

- `-trace-out DIR` exports CSV files from the collector: busy fractions per resource and time bin, tasks, chunks, marks, phases and the culprit tree.
- The raw writer (`-trace-raw DIR`) writes the raw dump for r2-vis (Section 3). Its configuration (`-trace-config FILE` and `-set trace.KEY=VALUE`) selects record types, subtrees and a time window. The filters decide at the observation point; they never buffer or reorder.

Each charge has an occupant: the task of a chunk; the task or program of a message; the invoked task for the cost $c_{desc} + c_{invoke}$ of its call; the program for other scalar work, loads and stores. Document 4 gives the records (Section 3).

11.6 Assembly listings

The option `-asm DIR` writes one listing per program instance, in the file

`DIR/<neutrino>.<k>.<program>.s`

with k the number of the instance on its Neutrino. A listing has one instruction per request of the program, in program order, with the cycle of the request in a comment. The input is the stream of requests, so a listing exists for any program, written in Rust or otherwise.

- A task prints as its opcode (`dma`, `mm`, `vec`, `join`) with `dst=<range>@v<n>`, `pe=`, `deps=[...]`, its arguments and its label. A range prints its span in bytes, then `/` and its mask when the mask is not all ones. `untracked` after the opcode marks an untracked task. `@v?` marks a versioned destination whose version the program did not infer.
- The other requests print as `spawn`, `wait`, `version`, `vreset_all`, `vreset`, `cycles`, `load`, `store`, `mark` and `cache_flush`.
- An address prints relative to the narrowest named range that holds it: a memory (a hashed memory at its linear range), a bank outside the range of its memory, the linear range of a hash link, or a sync range.

Listing 2: Listing of the program of the SM in the GEMM example of document 1 (Listing 5), shortened, with the labels.

```
.program root.gpc0.sm0 g    ; parent root.gpc0, started at cycle 122
    dma dst=root.gpc0.sm0.smem+0x0:0x2000@v1 pe=root.gpc0.sm0.dma deps=[] from=root.hbm+0x0 to=
    ↪ root.gpc0.sm0.smem+0x0 bytes=8192 tile=8192 label=864691128455135232    ; t=122
    dma dst=root.gpc0.sm0.smem+0x2000:0x2000@v1 pe=root.gpc0.sm0.dma deps=[] from=root.hbm+0
    ↪ x4000 to=root.gpc0.sm0.smem+0x2000 bytes=8192 tile=8192 label=864691128455135232    ; t
    ↪ =122
    mm dst=root.gpc0.sm0.smem+0x8000:0x3f80/0x7e@v1 pe=root.gpc0.sm0.atlas0 deps=[root.gpc0.sm0.
    ↪ smem+0x0@v1,root.gpc0.sm0.smem+0x2000@v1] a=root.gpc0.sm0.smem+0x0 b=root.gpc0.sm0.smem
    ↪ +0x2000 lda=32 ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=0 keep=1 tile=64
    ↪ x64 label=1008806316530991104    ; t=122
    ...
    dma dst=root.gpc0.sm0.sync+0x0:0x200@v1 pe=root.gpc0.sm0.dma deps=[root.gpc0.sm0.smem+0
    ↪ x8000@v2,root.gpc0.sm0.smem+0xc000@v2,root.gpc0.sm0.smem+0x8080@v2,root.gpc0.sm0.smem+0
    ↪ xc080@v2] from=root.gpc0.sm0.smem+0x8000 to=root.hbm+0x8000 bytes=32768 tile=32768 label
    ↪ =936748722493063168    ; t=122
    wait root.gpc0.sm0.sync+0x0@v1    ; t=122
```

11.7 Diagnostics

- Warnings do not stop a run: warnings of the tables, a race when the race check is set to warn, and a late consumer (a pending entry over a range that a completed task already wrote; off by default).
- At the end, the simulator lists every pending entry that is still open, with the last writer of its range (on by default). An open entry never released its waiters. The list shows a consumer whose producer was never invoked, or a consumer that came after its producer retired.
- It lists the programs that did not finish and what blocks each one: a wait, a stall or a request.
- A deadlock gives the occupancy of every table and every stage that holds messages (Section 4.9).

The option `-no-diagnostics` turns off the list of open entries and the late-consumer check.

12 Extending the simulator

This section gives the procedures for the common extensions. Each extension keeps the separation of the planes (Section 2).

12.1 A new opcode

1. Write a profile function from a descriptor and a node configuration to a profile (Section 6). Obey Rule S6.1.
2. For a transfer, take the route halves from the routing tables. For a compute chunk, give one demand per resource, with its direction (read, write or work) and its utilization.
3. Keep the work per chunk constant and small: no search, hash or allocation per chunk. The number of chunks sets the number of events.
4. Register the function in the registry under a name. The registry gives the opcode. The assembly listing prints the name.
5. For new arguments, add them to the descriptor with their wire bytes (Section 4.3) and their form in the listing.
6. If the opcode writes data, add its effect to the Numeric and Trace backends. The task must write only inside its destination range.

12.2 A new kind of PE

1. Add a compute `type` to the loader, with its keys and checks. Give each rate in units per cycle of the clock of the element.
2. Add its flow resources, and attach it as a master to its buses.
3. Write the profiles of its opcodes.
4. Name its resources by the scheme of Table 11, so that the statistics and the trace find their owner node by name.

12.3 The programs and their builders

The crate `nu-prog` holds the programs (Table 3). The program `forward` runs the vision encoder, the VLM prefill and the action expert of pi0. The other programs are `prefill`, `adamoe`, `micro` (microbenchmarks) and `kernels` (GPU kernels for the RTX PRO 6000 reference, Section 13.3.2). The parameters of a program are a TOML file (`--params`) and changes (`--param key=value`). They are separate from the configuration of the device (Rule S10.1).

A **builder** is the Rust code of a program that makes its tasks. For one program, it does these steps:

1. It finds the elements of the device by name: the SMs (the nodes with the PEs `atlas`, `muon` and `dma`, a scratch memory `smem` and a sync-only space), the DRAMs (`hbm`) and the cluster memories (`cmem`).
2. It places every buffer at an address and checks the capacity of each memory (Section 3.2).
3. It cuts the work into tasks.
4. It builds the descriptors and their dependencies in **graph windows**: groups of tasks that it plans before it invokes them.

Example. The builders of the 2p family place a tensor in the cluster DRAM of the cluster whose SMs read it most. They spread the weights of the prefill over the cluster DRAMs in blocks of 128 columns. In the SMEM, they put operand A in bank 0 and operand B in bank 1 (Section 3.1).

Drivers. In the program `forward`, each cluster driver invokes the tasks whose destination range is in the subtree of its cluster (Section 6.2). So an invoke never crosses a cluster, and a dependency between clusters is a registration on a key (Await). Every driver plans the whole graph window and counts the versions of every task of the graph window. It invokes only its own tasks.

Dependencies and versions. The owner of a range gives a version to each write of the range, in the order in which the invokes arrive (Section 7). A dependency names a key: an address and a version. A builder makes these keys from addresses. It builds the tasks of a graph window with these rules:

1. Every task writes a versioned range. A `join` writes the next range of a ring of 32 ranges in the sync-only space of its owner.
2. The builder names a dependency by an address that the consumer reads. The address resolves to the latest earlier task of the graph window whose destination range holds it. The dependency is the key of that task: the base of its destination range and its version.
3. A writer also depends on the previous version of its destination range in the graph window (write after write). A writer that reuses a range depends on the outputs of the readers of the old data (write after read, Section 3).

4. A graph window ends with a barrier. A dependency on a producer of an earlier graph window is dropped, because the barrier orders it. A producer that the builder carries into the next graph window is the exception.
5. A local invoke returns its version. For a forwarded invoke, the driver infers the version: it counts the invokes to each base address. The count is exact because one driver invokes all producers of a range, and its invokes reach the owner in program order (Rule R3.10). A debug build of r2-sim checks each inferred version at the owner. With several drivers, each driver counts the tasks of all drivers, in program order.
6. The versioned ranges of a graph window must fit the version tables of their owners. Before the first invoke of a graph window, the driver releases (`vreset`) the signals of the last graph window. When the table of an owner would overflow, it also releases the least recently used range that is done. A release starts the count of that base again, at the owner and in the driver.
7. A **pin** names the current producer of a range for a consumer that the builder adds later, even when another task writes the range in between. A signal with one dependency is a pin: the builder invokes no `join` for it.
8. A forwarded descriptor must fit in 4 beats (Rule N4.3). A task with more dependencies gets a `join` that takes the extra dependencies. One `join` takes at most 29 dependencies.

12.4 A new program

1. Write an asynchronous function over `NuCtx`. Obey Rule S4.2.
2. Register it in the program launcher with its parameters (a TOML structure that rejects unknown keys).
3. Infer the version of each forwarded invoke and write it into the descriptor. A debug build then checks each inference.

Note. These conditions are frequent causes of a deadlock or of a wrong version in a new program (Section 7 gives the rules of versions):

- Each reader of a version must be invoked before the next producer of the same range.
- Two writers of one range from different Neutrinos need an explicit order: the owner numbers the versions in the order in which the invokes arrive.
- A version base must keep one length in every task.
- Streams that share the tables of one owner deadlock when their tasks in flight exceed what the owner holds. Keep the tasks in flight within the scoreboard minus the issue headroom.

12.5 A new backend or tracer

A backend implements five functions: configure (the memory blocks and the sync-only ranges), has data, apply an op, read, and write before the run. It must apply the ops in the order of their sequence numbers (Rule S3.1). A read must return the state after the ops that the worker applied.

A tracer implements the observation points that it needs. It must obey Rule S11.1: it returns nothing and the simulator reads nothing from it. It says at the start of the run whether it needs the charges, so that the resource plane records them only then.

12.6 Checks of a change

- The test suites of `nu-model`, `sim-core` and `nu-prog` compute the expected cycles by hand on small configurations. A change that moves such a value must explain the move by its mechanism.
- A set of reference runs of the forward pass (the golden runs) checks that a change that must not change the time leaves every end cycle and every statistics file the same (`tools/golden.sh`).
- A new field of the statistics file must not appear when it is empty, so that the reference files stay the same.

13 Examples

The examples come from real runs of r2-sim (the working tree of 2026-10-15) on the RTX PRO 6000 configuration of Section 13.3. They show the mechanisms of the earlier sections on small cases. They are not measurements of a device. The script `r2sim/examples/run.sh` next to this document repeats these runs and the runs of Sections 4.7 and 8.3.

13.1 One DMA through a route

The `micro` program in mode `latency` with the driver on an SM copies 1 KiB from the L2 into the SMEM of SM `root.gpc0.sm0`:

```
r2sim run --config configs/rtx6000.yaml --program micro \
    --param 'mode="latency"' --param bytes=1024 --param 'mem="l2"' \
    --param 'driver="sm"' --trace-raw DIR
```

Task. The program calls `invoke` at cycle 0. The descriptor costs $c_{desc} = 15$ and $c_{invoke} = 8$ cycles on `root.gpc0.sm0.scalar` (Section 4.3). At cycle 23 the task enters the tables, is ready and issues on the copy engine `root.gpc0.sm0.dma`.

Chunks. The source is in the linear range of the hash link of the L2. The hash granule is 256 bytes, so the transfer has four chunks of 256 bytes (Rule S5.4). The linear addresses `0x000`, `0x100`, `0x200`, `0x300` go to the slices `l2.b0`, `b1`, `b0`, `b1`. The engine has a beat of 32 bytes at clock 1, so it issues one chunk every 8 cycles: at 23, 31, 39 and 47. The pipeline fill is the latency of the L2 (142) and the engine latency is 289. So every chunk has the earliest cycle $\max(issue, 23 + 142) + 289 = 454$.

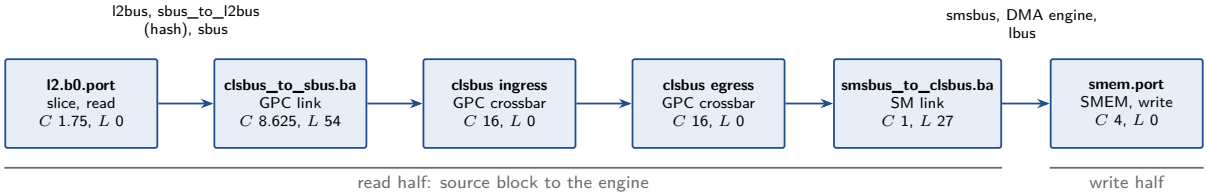


Figure 8: The route of a chunk from an L2 slice to the SMEM of an SM in the RTX PRO 6000 configuration. Boxes are rated hops with their capacity C in slots and their latency L. Gray labels are unlimited elements; their latency is 0.

Route. Figure 8 shows the route. The read half goes from the slice to the engine. It has the read port of the slice, the GPC link down, the two ports of the GPC crossbar and the link of the SM down. The unlimited buses `l2bus` and `sbus`, with the hash link between them, add no hop. The write half goes from the engine over the unlimited `lbus` to the port of the SMEM.

Chunk 0. The chunk needs one slot on each hop: the master rate is 32 bytes per cycle and every beat is 32 bytes (Rules S5.6 and S5.7).

Hop	C	L	u	Window
`root.l2.b0.port`	1.75	0	1	[454, 462)
`root.gpc0.clsbus_to_sbus.ba`	8.625	54	1	[454, 462)
`root.gpc0.clsbus.clsbus_to_sbus.in`	16	0	1	[508, 516)
`root.gpc0.clsbus.sm0.smsbus_to_clsbus.out`	16	0	1	[508, 516)
`root.gpc0.sm0.smsbus_to_clsbus.ba`	1	27	1	[508, 516)
`root.gpc0.sm0.smem.port`	4	0	1	[535, 543)

Each hop starts at the start of the hop before it plus its latency (Rule S5.5): $454 + 54 = 508$ and $508 + 27 = 535$. On the slice port, 8 beats at one slot of a capacity of 1.75 take 8 cycles (Rule S5.2). The chunk ends at 543. All windows have 8 cycles, so the culprit is the first hop, the slice port.

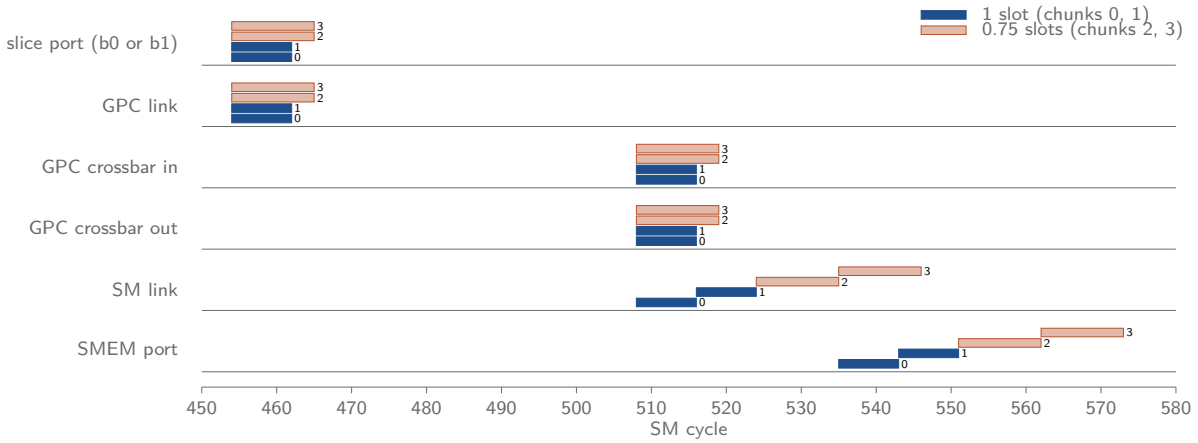


Figure 9: The windows of the four chunks of the copy on each hop. Numbers are the chunks. The bars of chunks 2 and 3 hold 0.75 slots.

Chunks 1 to 3. Figure 9 shows the four chunks.

- Chunk 1 goes to slice 1, so its first four hops have the same windows as chunk 0. The link of the SM has one slot, and chunk 0 holds it in $[508, 516)$. Window A is $[516, 524)$. No slot is free before 516, so there is no window B. The data waits 8 cycles between the crossbar port and the link (Rule S5.9). The SMEM port takes it in $[543, 551)$. The queue culprit is the link of the SM.
- Chunk 2 goes to slice 0 again. Chunk 0 holds one of the 1.75 slots in $[454, 462)$. Window A is $[462, 470)$. Window B starts at 454 on the 0.75 free slots: 19 slots of the resource, so $[454, 465)$. Window B ends first (Rule S5.8). The chunk now moves 24 bytes per cycle, so it needs 0.75 slots on every later hop (Rule S5.7): the GPC link $[454, 465)$, the crossbar ports $[508, 519)$, the link of the SM $[524, 535)$ and the SMEM port $[551, 562)$.

- Chunk 3 does the same on slice 1. It takes the link of the SM in $[535, 546)$ and the SMEM port in $[562, 573)$.

The last chunk ends at 573. The engine runs one task less, the completion port charges $c_{complete}$ in $[573, 577)$, and the task completes at cycle 577.

Note. The copy pays its latency once: all four chunks start at 454. The link of the SM, with one slot of 32 bytes, puts the chunks in a row. Chunks 2 and 3 keep the rate of 0.75 slots from the slice port on every later hop (Rule S5.7). This is also true on the link of the SM after chunk 1 has left it. Their data waited at the crossbar, so a real link would move it at its full rate. This is the cost of a constant usage per chunk.

13.2 One GEMM CTA as tasks

The `kernels` program runs a GEMM of $128 \times 128 \times 128$ in bf16 as one CTA on one SM, with the tile of a CUTLASS kernel: $b_m = b_n = 128$, $b_k = 32$ and 3 stages. The tensors are in the GDDR7. The Neutrino costs `c_invoke`, `c_base`, `c_dep` and `c_word` are 0, as in the validation runs, and `-acc-buffer 65536` gives each tensor unit an accumulator of 64 KiB. The parameters (TOML):

```
cache = "home"
sms = 1
tensors = [{name = "a", bytes = 32768}, {name = "b", bytes = 32768},
           {name = "c", bytes = 32768}]
kernels = [{name = "gemm", op = "gemm", inputs = ["a", "b"], output = "c",
            m = 128, n = 128, k = 128, bm = 128, bn = 128, bk = 32,
            stages = 3, ctas_per_sm = 1}]
```

The CTA as tasks. The CTA has 4 warps of 64×64 (the default $b_m b_n/4096$). Warp w runs its matmuls on the tensor unit `atlas`i with $i = w \bmod 4$. For each of the 4 k-tiles, the CTA has:

- a DMA of the A tile (128×32 elements, 8 KiB) and a DMA of the B tile (32×128, 8 KiB) from the GDDR7 into the buffers of stage $kt \bmod 3$ in the SMEM;
- one matmul per warp, $64 \times 64 \times 32$, that depends on the two DMAs and on the matmul of the same warp in the k-tile before. It accumulates from the second k-tile on, and it keeps C in the accumulator until the last k-tile.

The DMAs of k-tile 3 reuse the buffers of stage 0, so they depend on the four matmuls of k-tile 0 (the order of write after read is the program's job). The last task is the store: a DMA of the C tile (32 KiB) from the SMEM to the GDDR7 on the copy engine of the SM. Its destination range is a store record in the sync space of the SM (Section 13.3.2). The program of the SM invokes the 25 tasks and waits for the store record (Listing 2). The device spawns a program on the cluster, which spawns the program of the SM, and marks `gemm start` and `gemm end`.

Timeline. Table 22 gives the cycles of the tasks.

- The six DMAs of the first three k-tiles issue together at 122. Their chunks are 256 bytes (the granule of the hash of the GDDR7 channels). Each chunk starts at or after $122+142+289 = 553$. All of them share the link of the SM at 32 bytes per cycle, which is the queue culprit of most of their chunks.
- A matmul of k-tile 0 has one chunk: $64 \times 64 \times 32 = 131072$ MACs at 128 MACs per cycle, so 1024 cycles, $[2202, 3226)$. Its A and B tiles, 8 KiB in 256 beats of 32 bytes, hold $256/1024 = 0.25$ slots of the SMEM port (4 slots) for 1024 cycles (Section 5.7). The four tensor units together hold one slot.

Table 22: The tasks of the GEMM CTA.

Tasks	PE	Ready	Issue	Complete
DMA A and B, k-tiles 0 to 2 (6 tasks)	`dma`	122	122	2198 to 2250
matmuls of k-tile 0	`atlas0` to `atlas3`	2202	2202	3230 to 3242
matmuls of k-tile 1	`atlas0` to `atlas3`	3230 to 3242	3230 to 3242	4258 to 4270
DMA A and B, k-tile 3	`dma`	3242	3242	4304, 4308
matmuls of k-tile 2	`atlas0` to `atlas3`	4258 to 4270	4258 to 4270	5286 to 5298
matmuls of k-tile 3	`atlas0` to `atlas3`	5286 to 5298	5286 to 5298	6314 to 6326
store of C	`dma`	6326	6326	7759

- The four matmuls end at 3226. The completion port of the Neutrino takes 4 cycles for each completion, so they complete at 3230, 3234, 3238 and 3242.
- The matmuls of the last k-tile also write C: 8 KiB per warp to the SMEM. The store then moves 32 KiB over the link of the SM and ends at 7759.

13.3 The RTX PRO 6000 as a modeled system

r2-sim can model a device that is not Radiance 2, if the device fits its terms: nodes, Neutrinos, memories, buses, links and PEs. The RTX PRO 6000 Blackwell (GB202) is the reference GPU for the correlation of the model. This section shows how its configuration and its programs map the GPU onto the model.

13.3.1 Hierarchy and parameters

Figure 10 shows the configuration (`rtx6000.yaml`, `rtx6000-gpc.yaml`, `rtx6000-sm.yaml`). One cycle is an SM cycle. Table 23 gives the parameters.

Table 23: Parameters of the RTX PRO 6000 configuration.

Element	Model
hierarchy	the device, 12 clusters (GPCs) of 16 SMs: 192 SMs for the 188 SMs of the card
L2	a scratch memory of 128 MiB as 64 banks (slices) of 2 MiB behind the hash link `sbus_to_l2bus` with the measured slice hash (six masks); per L2 cycle 64 bytes read and 32 bytes written in 32-byte beats; clock 7/8; latency 142; a cache of the GDDR7 with lines of 256 bytes
GDDR7	a DRAM of 64 banks (channels) behind the hash link `sbus_to_mbus`, which uses the slice hash of the L2, so a line has the channel of its slice; 2 bytes per DRAM cycle in 32-byte beats; clock 51/10; latency 142
fabric	the device crossbar without a limit; a GPC link of 276 bytes per cycle in 32-byte beats, latency 54; the GPC crossbar at 512 bytes per cycle per port; an SM link of 32 bytes per cycle, latency 27; control links with the same latencies
SM	an SMEM of 128 KiB at 128 bytes per cycle in 32-byte beats, latency 34; four tensor units `atlas0` to `atlas3` at 128, 256 and 512 MACs per cycle for bf16, fp8 and mxfp4 (one per sub-partition); a Muon of 128 lane operations per cycle, overhead 20; a copy engine (the TMA) with latency 289, 8 transfers in flight, 32-byte beats
device engine	a copy engine at the root: 64 transfers in flight, 512-byte beats; it carries the write-backs of the cache
Neutrinos	the tables of Radiance 2; the device has large tables so that they never limit a run

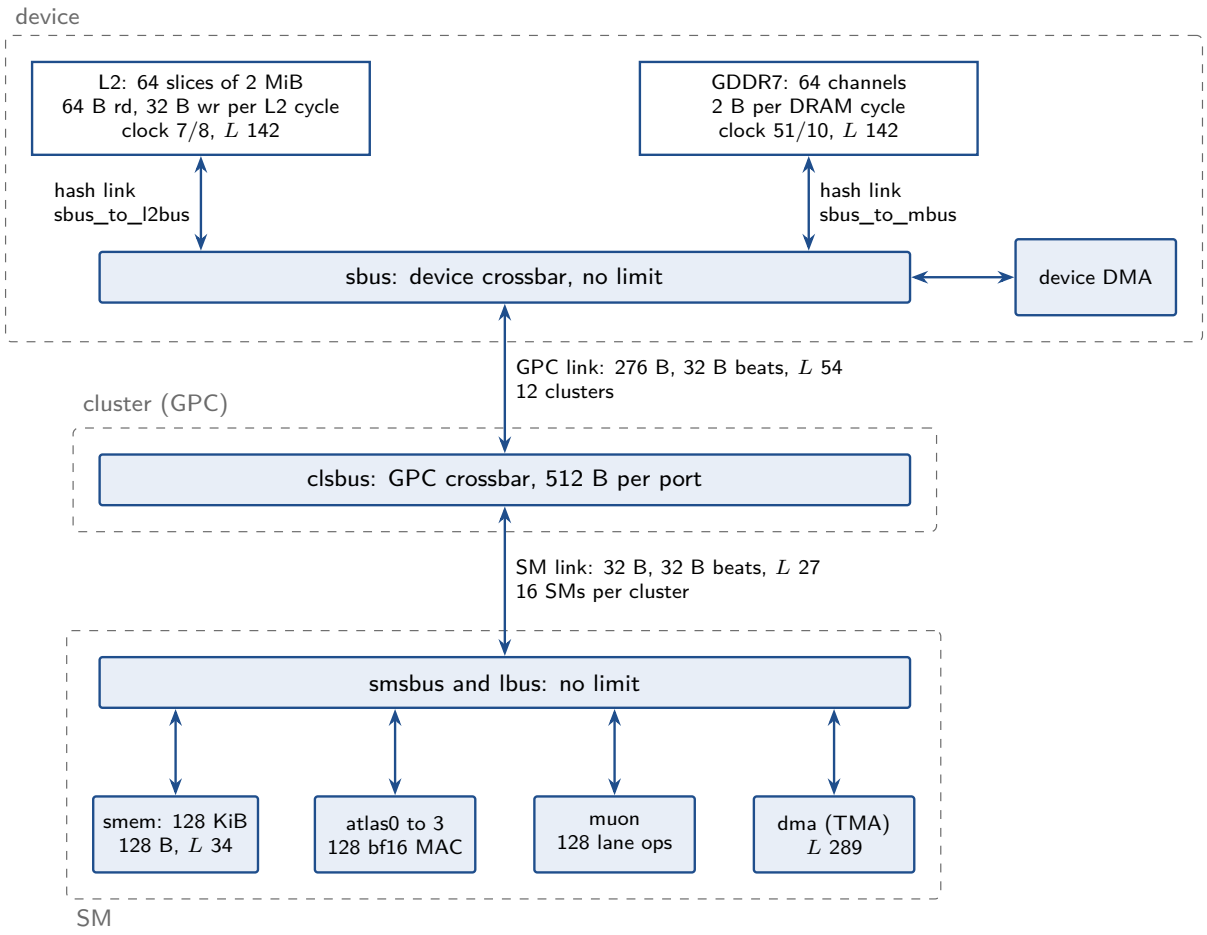


Figure 10: The RTX PRO 6000 configuration: the device with the L2 and the GDDR7 behind hash links, one of 12 clusters (GPCs) and one of its 16 SMs. Rates are per cycle of the own clock of each element.

Rule S13.1 (physical values). Every value of a reference configuration must be a number of the hardware or the result of a test that isolates it. No value must be set to make another test match.

Note. The latency of the copy engine of an SM is the one calibrated value of the copy path. It covers the way of the request to the L2 (the model charges a DMA only on the route of its data) and the time of the copy unit itself.

13.3.2 GPU semantics in the model

Table 24 maps the concepts of the GPU onto the model. The `kernels` program builds the tasks from a list of kernels with the tiles of the card's kernels.

Stores. On the card an SM stores directly into global memory. In Radiance 2 such a store is an untracked task (Section 5.4). It is a DMA task on the engine of the SM that writes into the memory of another node. No other task can depend on it. A tracked copy into the memory of another node is a pull by the engine of the owner (Section 5.1). r2-sim has untracked tasks (Section 4.2), but the `kernels` program keeps a convention for its stores. The store is a tracked DMA task on the copy engine of the SM. Its destination range is a **store record**: 512 B of

Table 24: GPU concepts and their model.

GPU	Model
kernel launch	per kernel, the device program spawns one program per cluster, which spawns one per SM, and waits for all of them; marks `<kernel> start` and `<kernel> end`
CTA	a list of tasks that the program of one SM invokes; CTA i runs on SM i mod n, with the SMs taken round-robin over the clusters
CTAs per SM	`ctas_per_sm` **CTA positions** per SM; each position has an equal part of the SMEM; a CTA starts when the CTA `ctas_per_sm` before it on the SM has completed its compute
warp tile on a tensor core	the matmuls of warp w run on tensor unit w mod 4; warp w is at warp row w mod m and warp column $\lfloor w/m \rfloor$ of the CTA tile, for m warp rows, as in CUTLASS; each warp writes its part of the C tile as a masked destination range (Section 2.3); when a mask cannot describe the part, for example at a granule of 512 B, each warp gets a base of its own, one granule from the next, and the ranges overlap
accumulator registers	the accumulator buffer of each tensor unit (`acc_buffer`), shared by the CTAs of the SM; `keep` until the last k-tile
asynchronous copy (TMA)	a DMA on the copy engine of the SM into a buffer of the SMEM
multistage pipeline	`stages` buffers; the loads of a stage depend on the matmuls that read it before
store of an output tile	a DMA on the copy engine of the SM from the SMEM to the L2 or the GDDR7; its destination range is a store record in the sync space of the SM
split-K semaphore	a dependency on the store record of the previous slice
SIMT instructions	a Muon task of lane operations at the utilization of its kernel variant (Section 9)
L2	the scratch memory of 64 slices, and one of the cache modes below
flush of the L2 between kernels	`cache_flush` before each kernel, or a cold L2 per kernel

the sync space of the SM per CTA position and output buffer. So the SM owns the task, and its Neutrino tracks only the completion of the store. The Neutrino of the device never sees it. Because the destination range is not the memory that the DMA writes, the program runs with the Null backend (Section 6.1).

The convention gives orders that an untracked task cannot give, so the program keeps it. Two orders depend on store records: the posted outputs below, and the split-K semaphore (Table 24). With untracked stores, each would need a fence (Rule N3.11). A fence waits for every untracked task of the program and blocks the program while it waits. So the order of one CTA position would become an order of the whole program, and the store of one CTA would no longer overlap the loads of the next CTAs. The split-K semaphore would also need a fence and a forwarded join per tile and slice.

Posted outputs. A CTA position has two output buffers. The next CTA in a position starts when the compute of the CTA before it is done. Its store can still be in flight. The producer of an output buffer waits for the store of the CTA two back in the same position.

L2 modes. The program has four modes for the L2 (`cache`):

- `first_touch` (default): the first read of a range, in launch order, comes from the GDDR7; later reads come from an L2 copy. Writes go to the L2. The program decides this when it

builds the tasks.
- `home`: every access goes to the home memory of its tensor.
- `per_kernel`: as `first_touch`, with an empty L2 at the start of each kernel. This stands for a measurement tool that flushes the L2 before each kernel.
- `sim`: every tensor is in the GDDR7 and every access goes there. The cache model (Section 8) decides what the L2 holds, and the device calls `cache_flush` before each kernel. The statistics give the counters of the cache per kernel.

SIMT. The SMs of the card have 128 lanes. The model has one Muon of 128 lane operations per cycle per SM. The utilization of each kernel variant comes from the method of Section 9.3.

Not modeled. The configuration records what it does not model:

- the limit of the card on the data in flight per GPC (the model uses a link rate);
- DRAM traffic through the ports of the L2 slices;
- the longer latency of a random dependent load from the GDDR7;
- the remote SMEM of a cluster of CTAs;
- the number of SMs: the model has 192 for the 188 of the card.

13.4 Runs of the listings of documents 1 and 2

The listings of document 1 (Section 10) and the forward-pass listing of document 2 (Listing 1) are assembly text from the runs of Table 25. Their files are in `docs/r2-spec/neutrino/listings/`.

Table 25: The runs that print the listings of documents 1 and 2. Each command runs in the directory `r2-sim` as `r2sim run --config C --program P [options] --asm DIR`.

Listings	C	P	Options
Listings 2 to 4	`rtx6000.yaml`	`kernels`	`--params layernorm.toml --acc-buffer 65536`; `c_invoke`, `c_base`, `c_dep`, `c_word` set to 0 on every Neutrino with `--set`
Listing 5	`rtx6000.yaml`	`kernels`	`--params gemm.toml`, the rest as for layernorm
Listing 6	`2p.yaml`	`micro`	`--param mode=pingpong --param rounds=2`
Listing 7	`2p.yaml`	`micro`	`--param mode=barrier --param sms=3 --param placement=spread`
Listing 8	`2p.yaml`	`micro`	`--param mode=invoke --param tasks=4`
Listings 9 to 11	`2p.yaml`	`micro`	`mode=stream mem=rsmem_gpc dir=write sms=2 placement=spread bytes=65536 chunk=16384 inflight=2 driver=sms`, each as `--param`
Listing 12	`2p.yaml`	`adamoe`	`--synthetic 1 --param steps=1 --param layers=1`
Listing 1	`2p.yaml`	`micro`	`mode=untracked sms=2 placement=spread bytes=65536 chunk=16384`, each as `--param`
Listing 1	`2p.yaml`	`forward`	`--params configs/forward.toml --param weight_dtype=mxfp4 --acc-buffer 1048576 --synthetic 7`